Informed Search - II

A* Search Algorithm



Previous Lecture

- Heuristics
- Greedy Search

A* Search

eval-fn: $f(n) = g(n) + h(n)$

A* (A Star)

- Although Greedy Search can considerably cut the search time (efficient), it is neither optimal nor complete.
- Uniform Cost Search minimizes the cost $g(n)$ from the initial state to n. UCS is optimal and complete but not efficient.

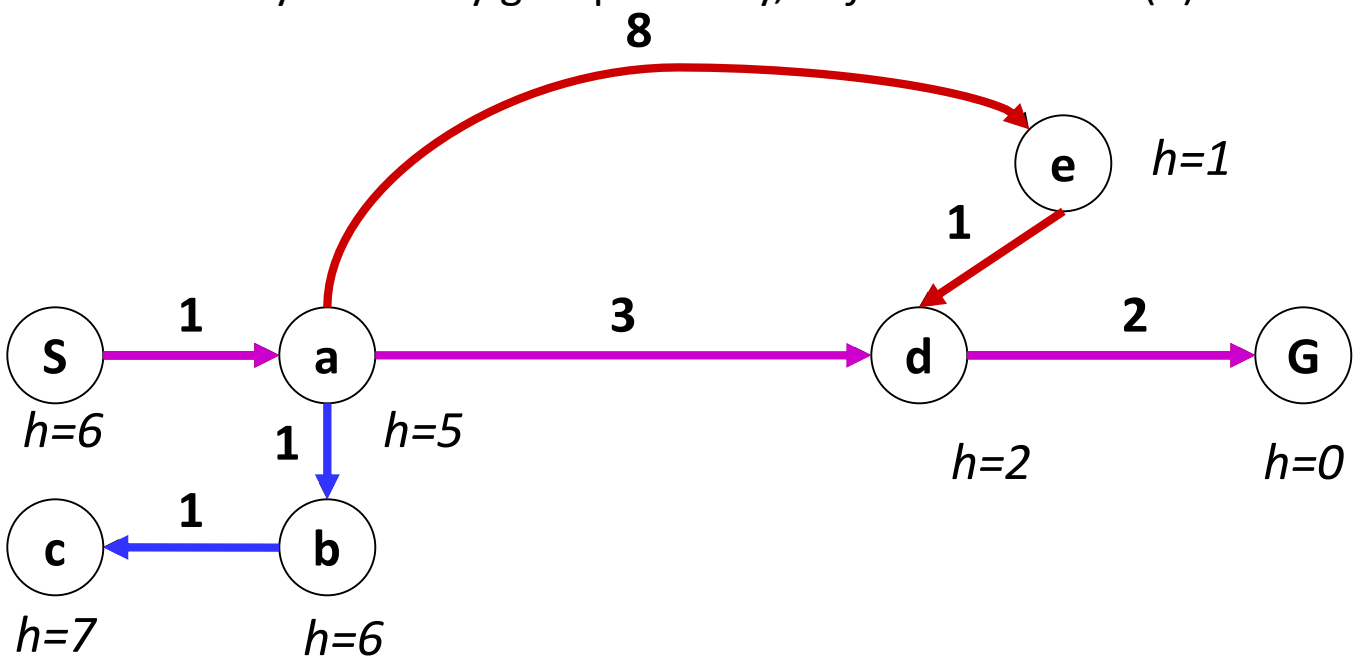
- New Strategy: Combine Greedy Search and UCS to get an efficient algorithm which is complete and optimal.

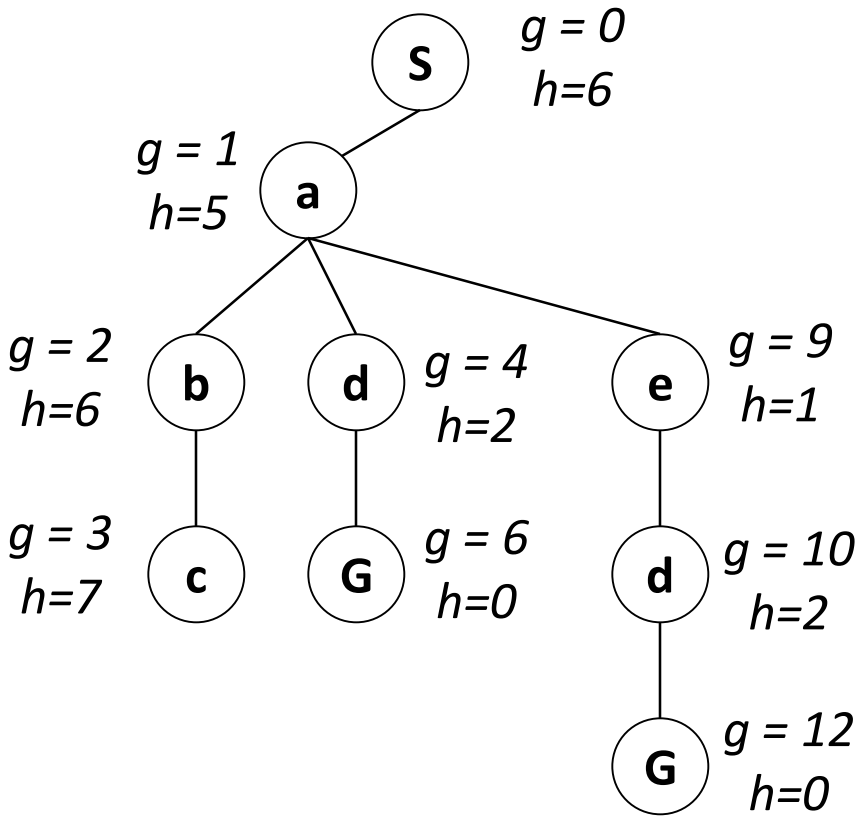
- A* uses a heuristic function which combines g(n) and h(n):

$$f(n) = g(n) + h(n)$$

Combining UCS and Greedy

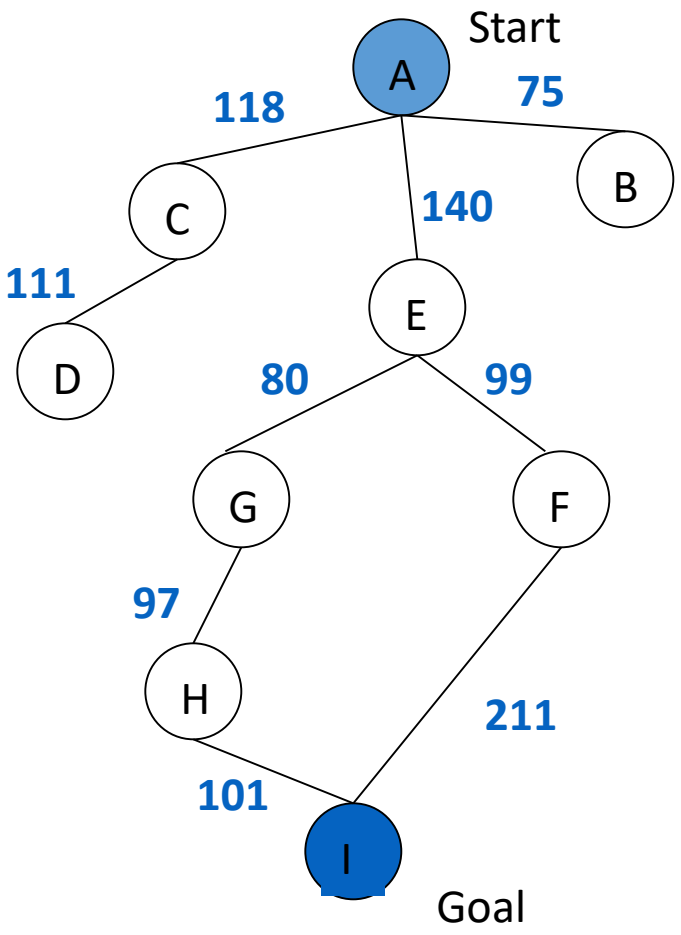
- Uniform-cost orders by path cost, or *backward cost* g(n)
- Greedy orders by goal proximity, or *forward cost* h(n)





- A* Search orders by the sum: f(n) = g(n) + h(n)

A* Search



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

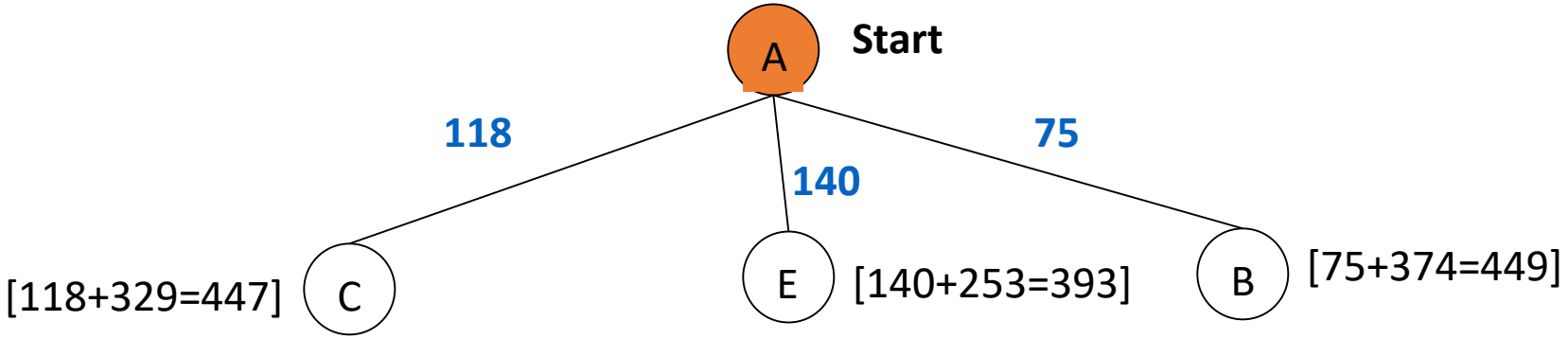
$$f(n) = g(n) + h(n)$$

g(n): is the exact cost to reach node *n* from the initial state.

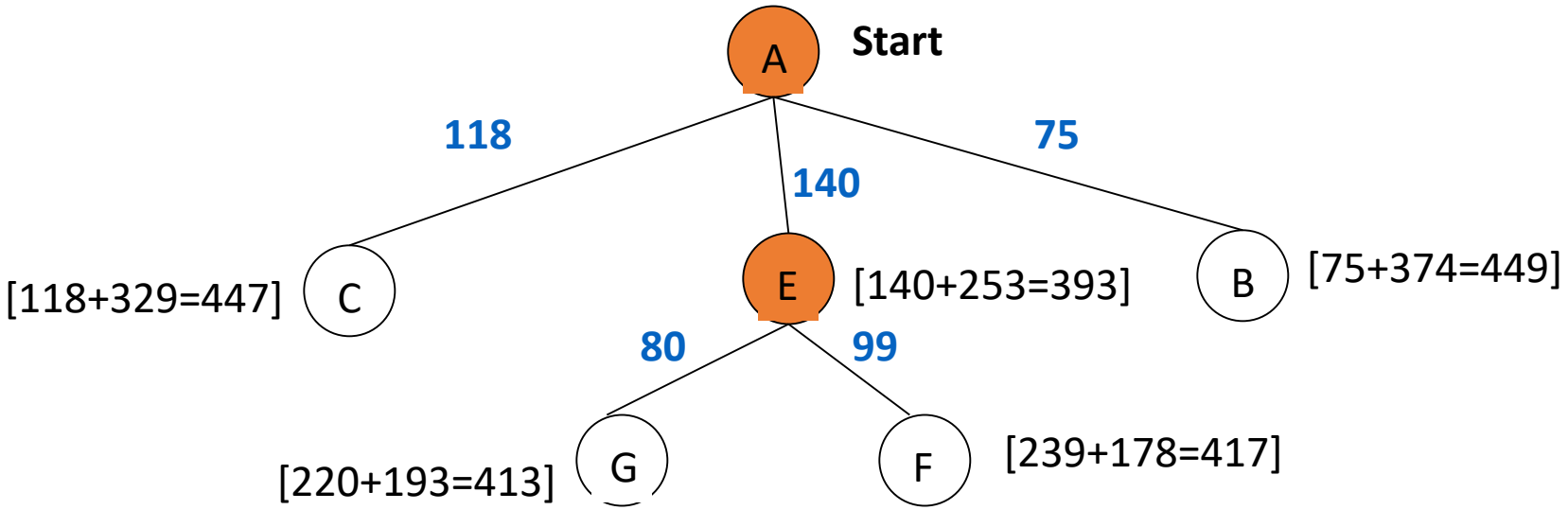
A* Search: Tree Search



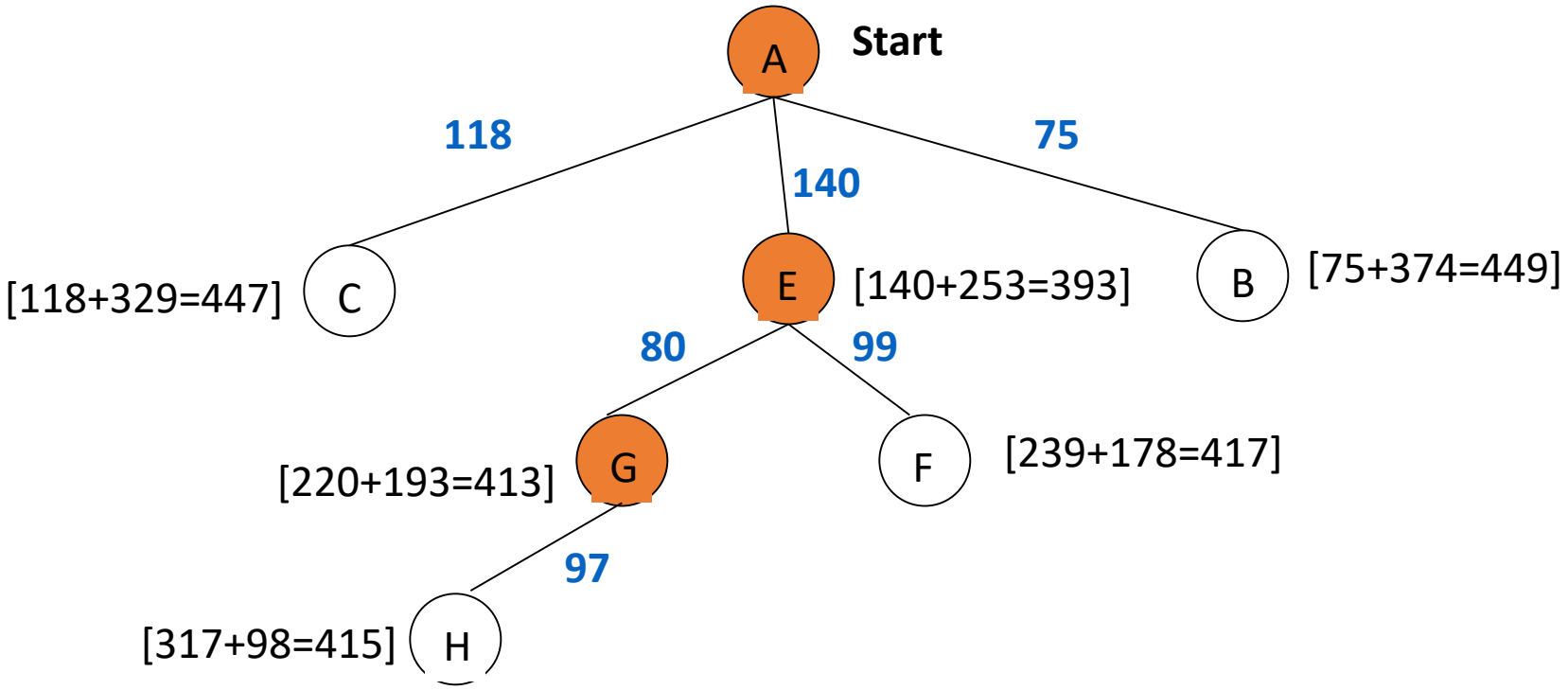
A* Search: Tree Search



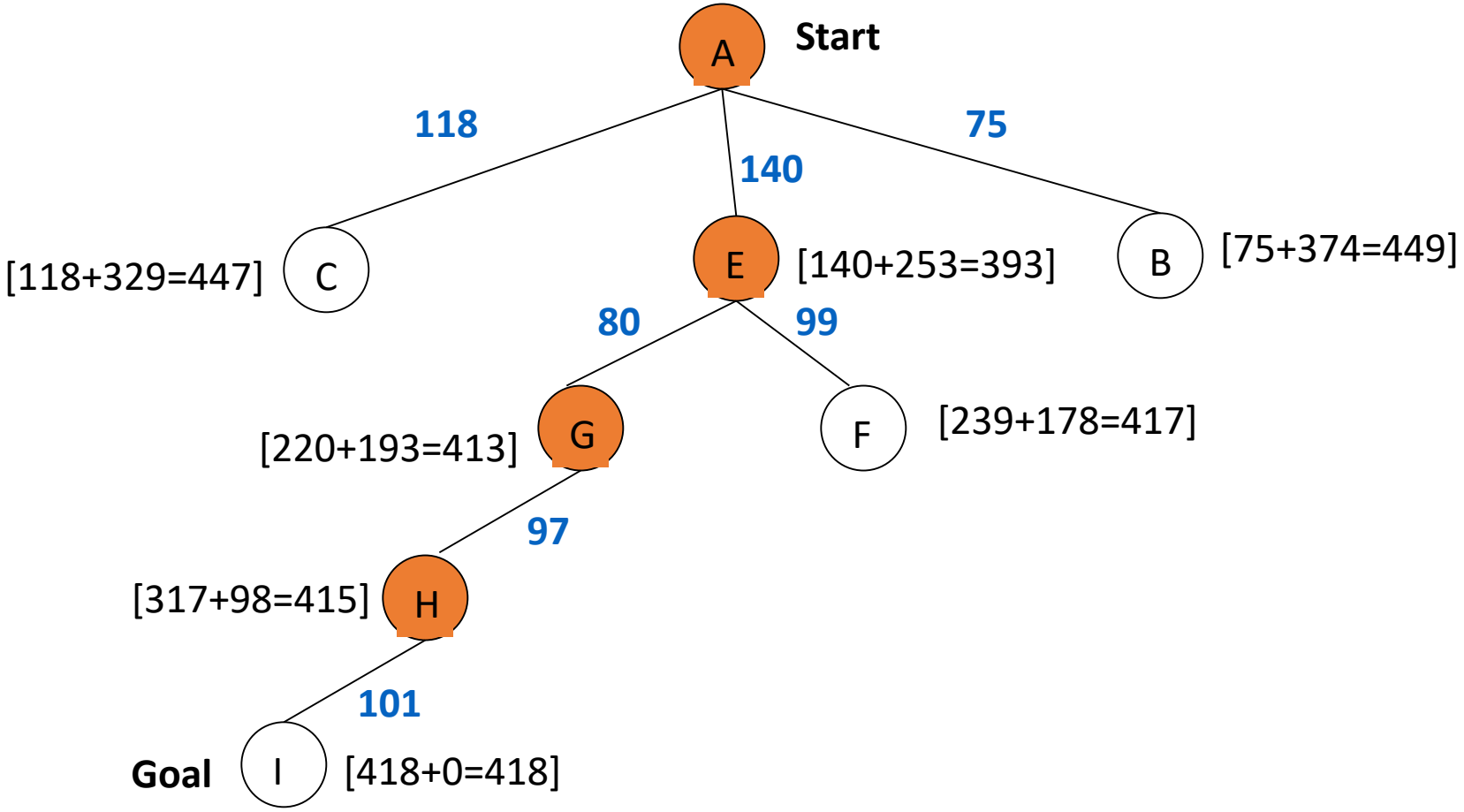
A* Search: Tree Search



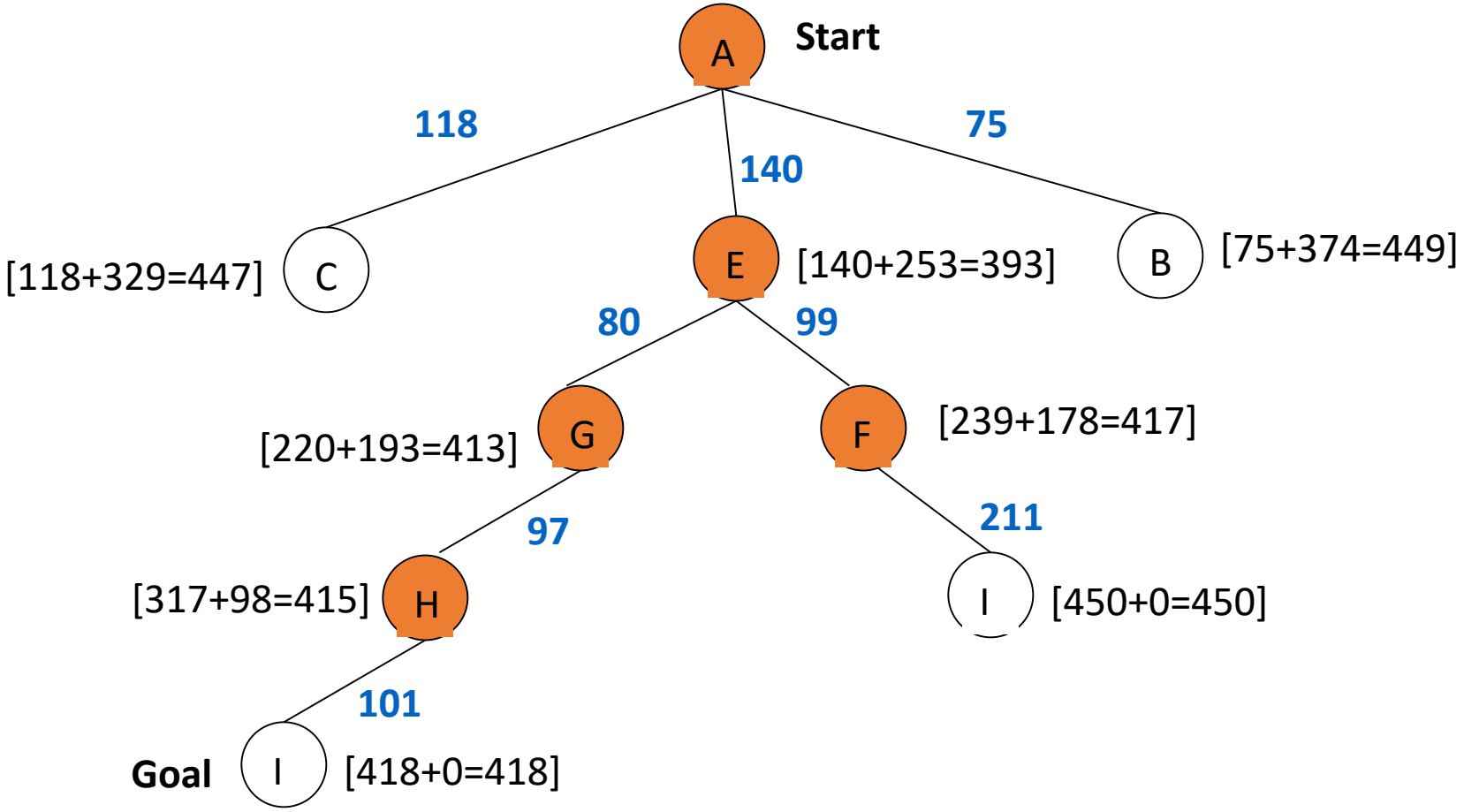
A* Search: Tree Search



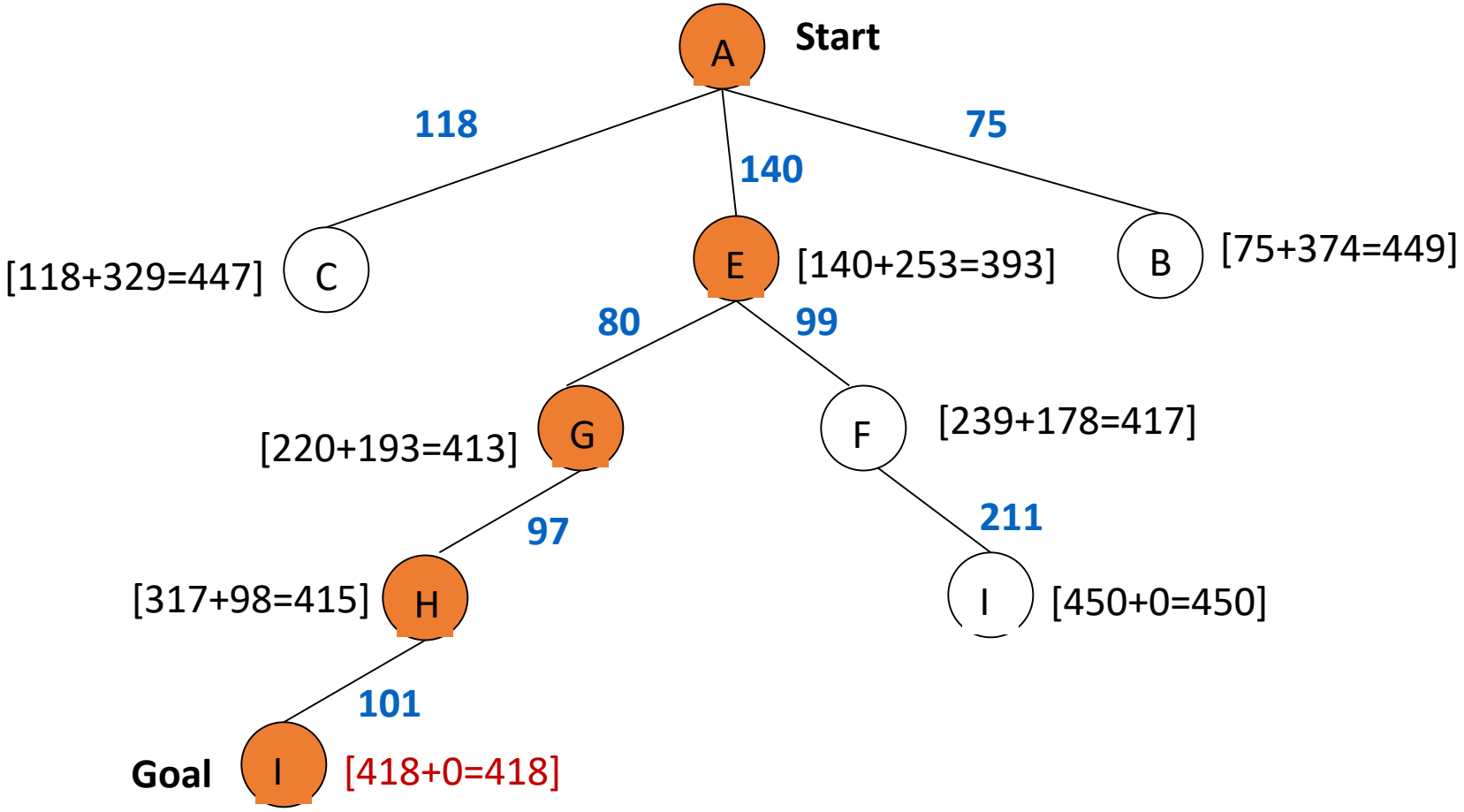
A* Search: Tree Search



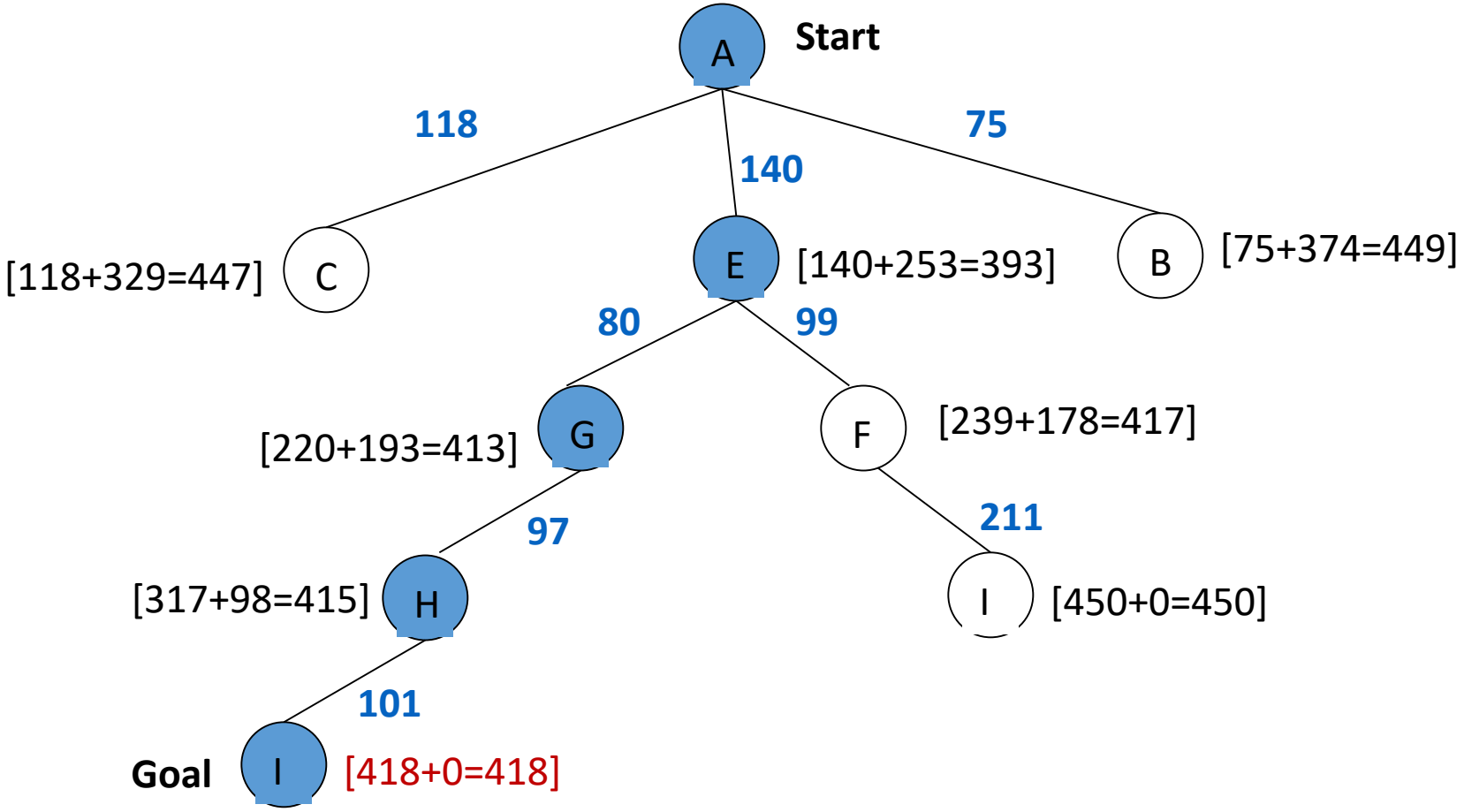
A* Search: Tree Search



A* Search: Tree Search

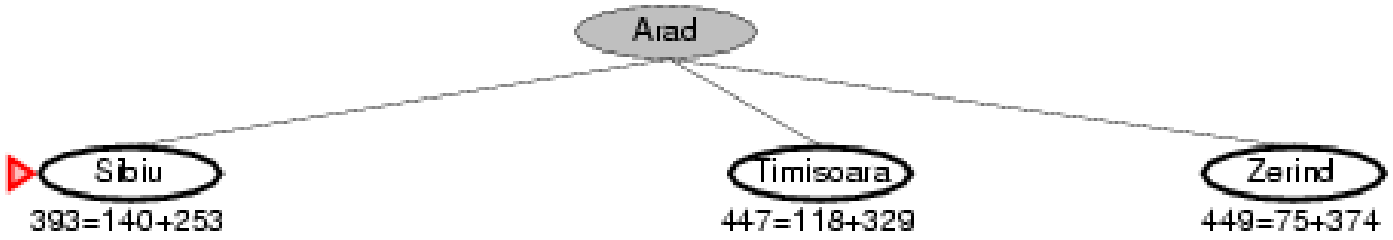


A* Search: Tree Search

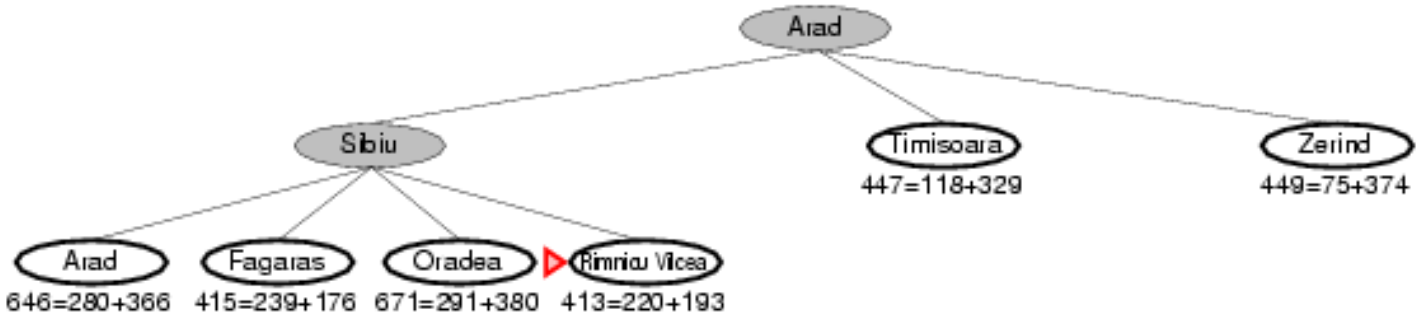


dist(A-E-G-H-I) =140+80+97+101=418

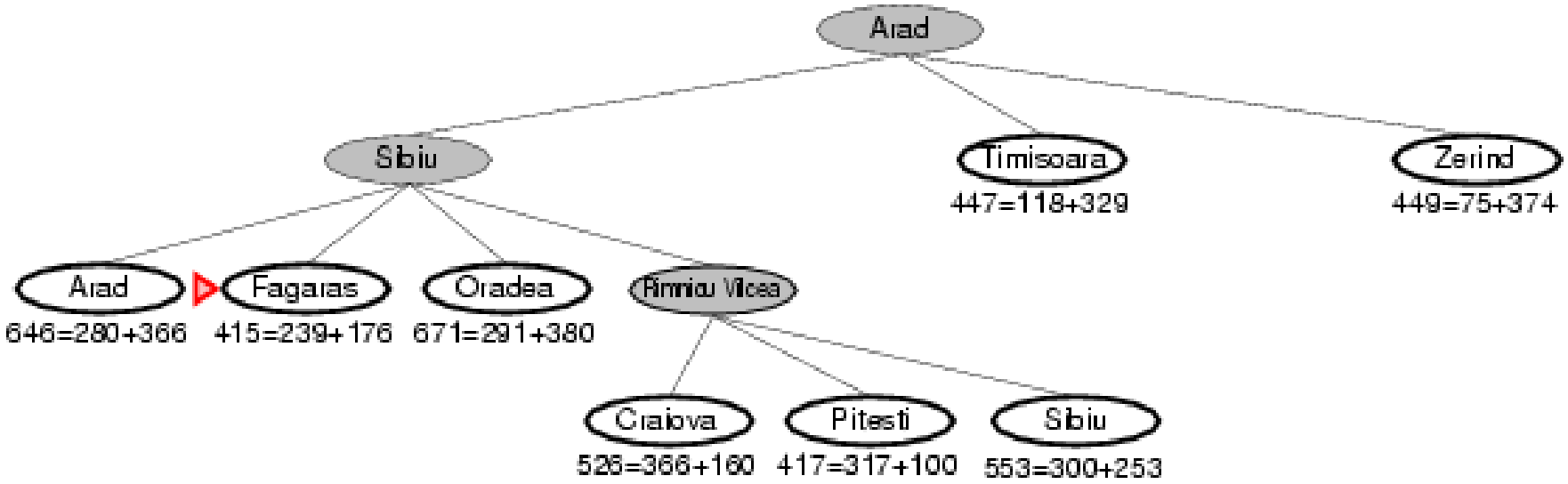
A* search example



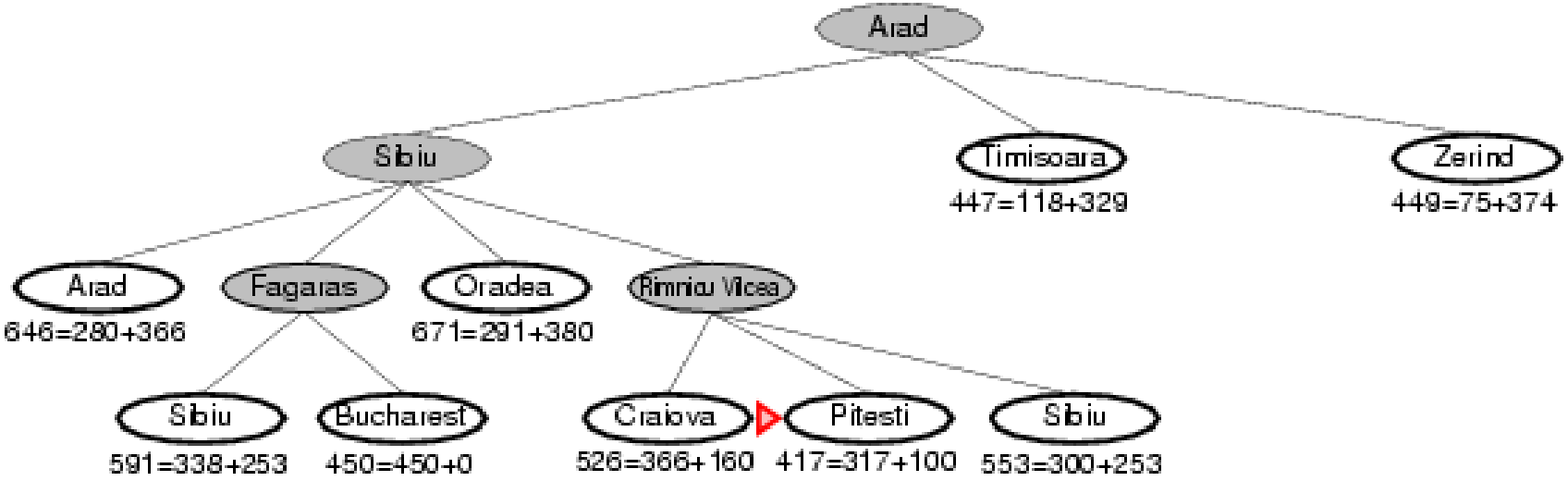
A* search example



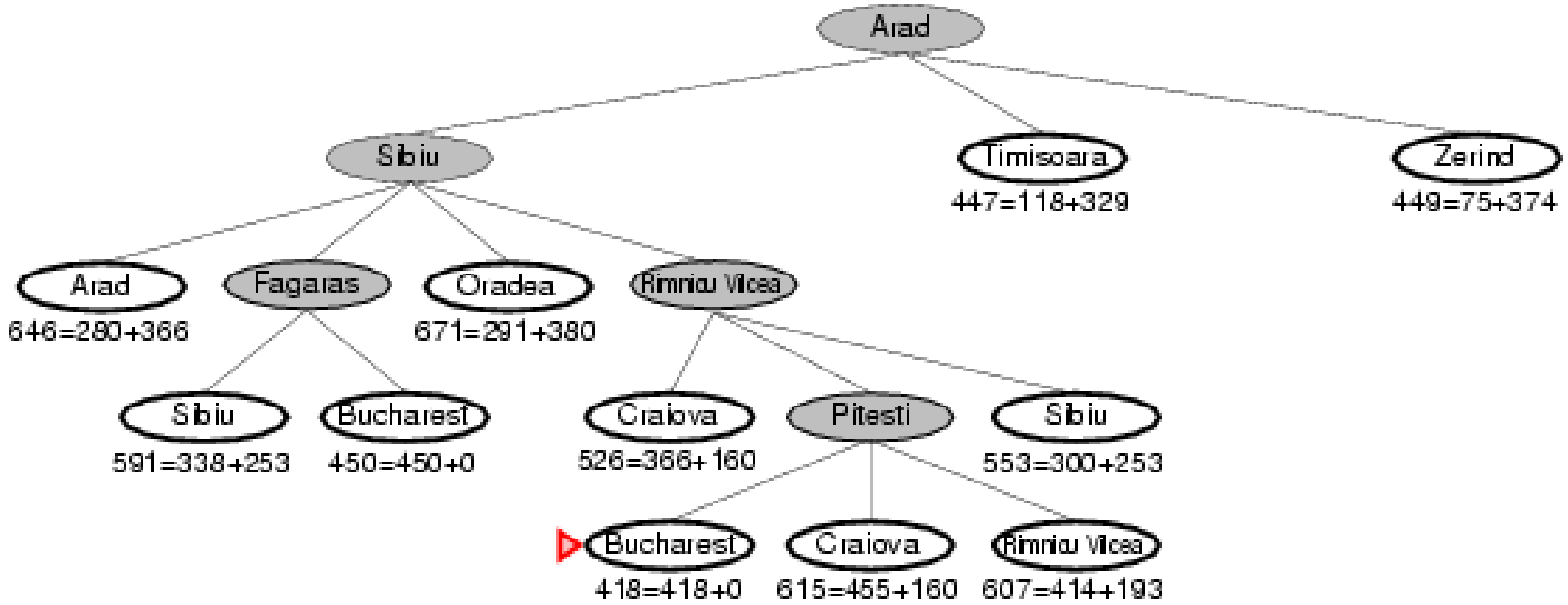
A* search example



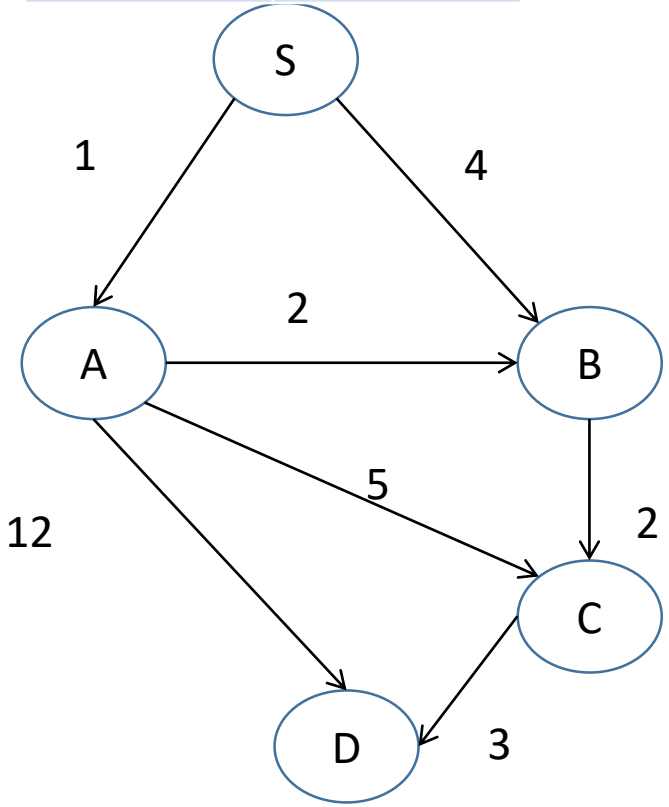
A* search example



A* search example

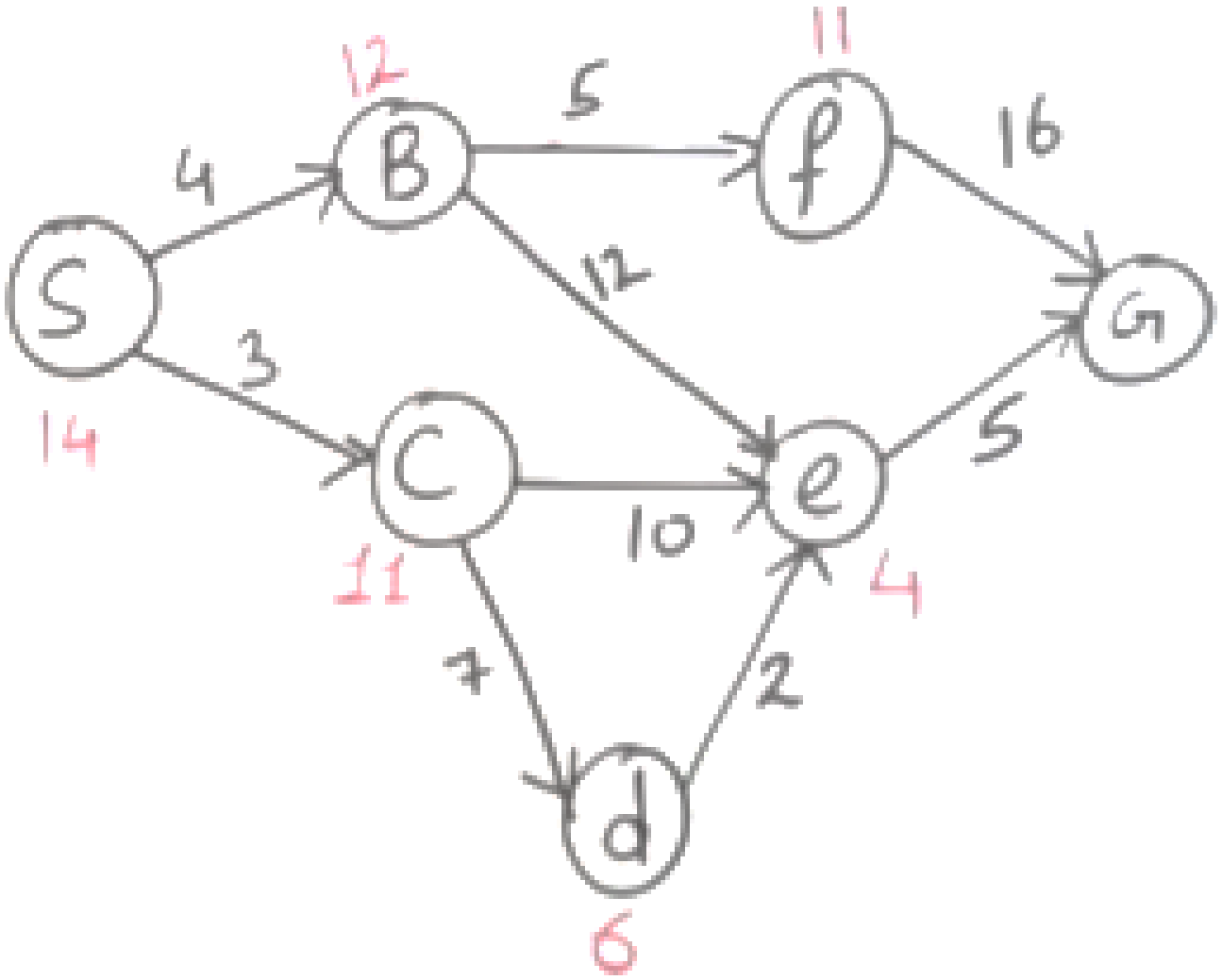


Heuristic Value	
S	7
A	6
B	2
C	1
D	0

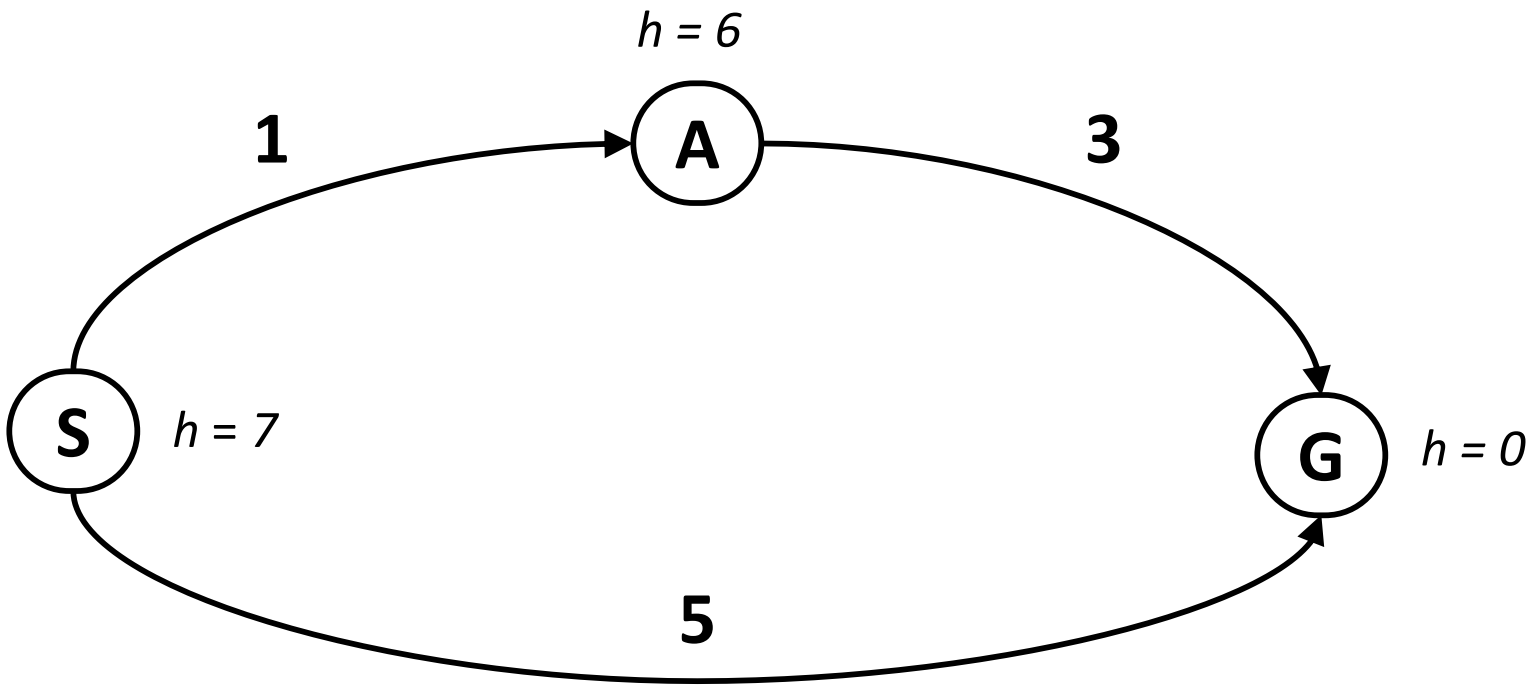


f(n) = g(n)+h(n)

	FRONTIER	EXPAND
1	(S,0+7=7)	S
2	(S-A,1+6=7)(S-B,4+2=6)	B
3	(S-A,7)(S-B-C,6+1=7)	A
4	(S-A-B,3+2=5)(S-A-C,6+1=7)(S-A-D,13+0=13)(S-B-C,7)	B
5	(S-A-B-C,5+1=6) (S-A-C,7)(S-A-D,13)(S-B-C,7)	C
6	(S-A-B-C-D,8+0=8) (S-A-C,7)(S-A-D,13)(S-B-C,7)	G


12
11
5
B
f
4
16
S
12
G
3
14
C
e
5
10
11
4
7
2
d
6

Is A* Optimal?



- What went wrong?
  - Actual bad goal cost < estimated good goal cost
  - We need estimates to be less than actual costs!

Admissible Heuristic Functions

- A heuristic $h(n)$ is admissible if $h(n)$ never overestimates the cost to reach the goal.
  - Admissible heuristics are “optimistic” approximation

- Mathematically *h* is admissible if

  For all n, $h(n) \leq C(n)$

where

- *n* is the node, *h* is heuristic
- *h(n)* is the cost indicated by *h* to reach goal from *n*
- *C(n)* is actual cost to reach a goal from *n*

Admissible heuristics Examples

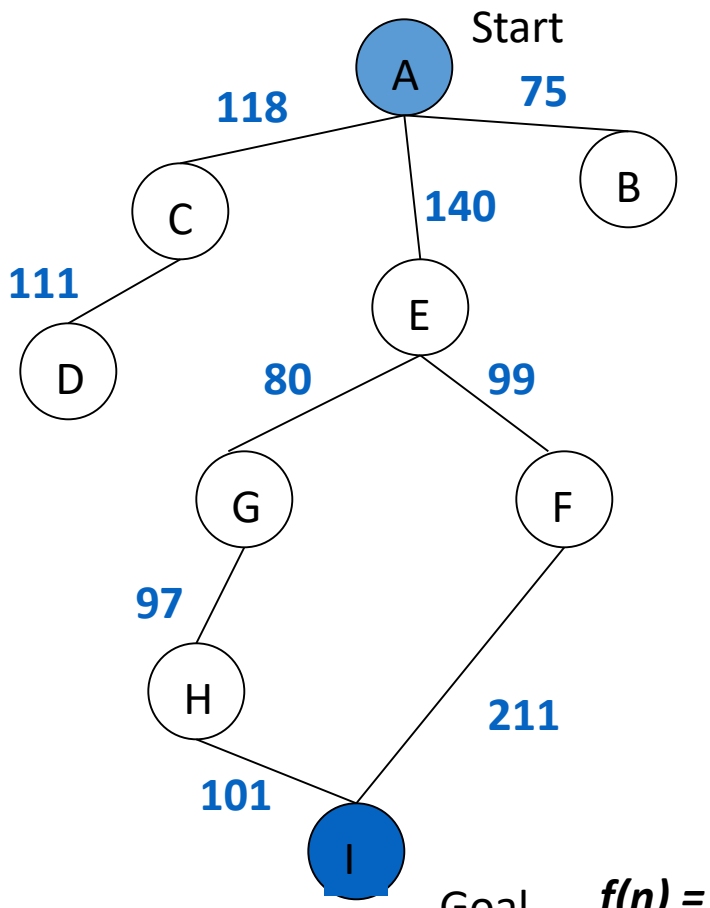
- $h_{SLD}(n)$ Straight line distance heuristic
  - Because never overestimate the actual road distance.
- Number of misplaced tiles in n-puzzle problem

Admissible Heuristic Functions

- Evaluation function $f(n) = g(n) + h(n)$ is admissible if $h(n)$ is admissible.
- However, $g(n)$ is the exact cost to reach node *n* from the initial state.
- Therefore, $f(n)$ never over-estimate the true cost to reach the goal state through node *n*.

- **Theorem**: If *h(n)* is admissible, A* using `TREE-SEARCH` is optimal

A* Search: if h not admissible



State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	**138**
I	0

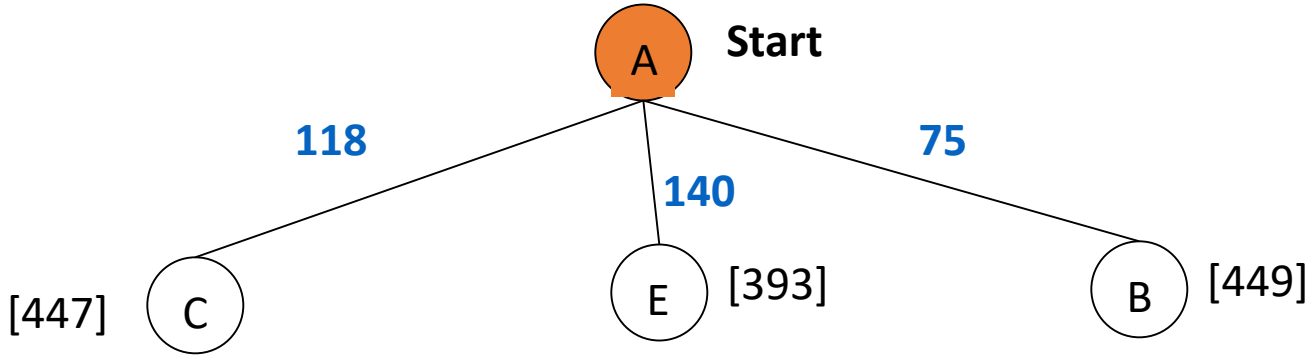
f(n) = g(n) + h (n) **– (H-I) Overestimated**

g(n): is the exact cost to reach node *n* from the initial state.

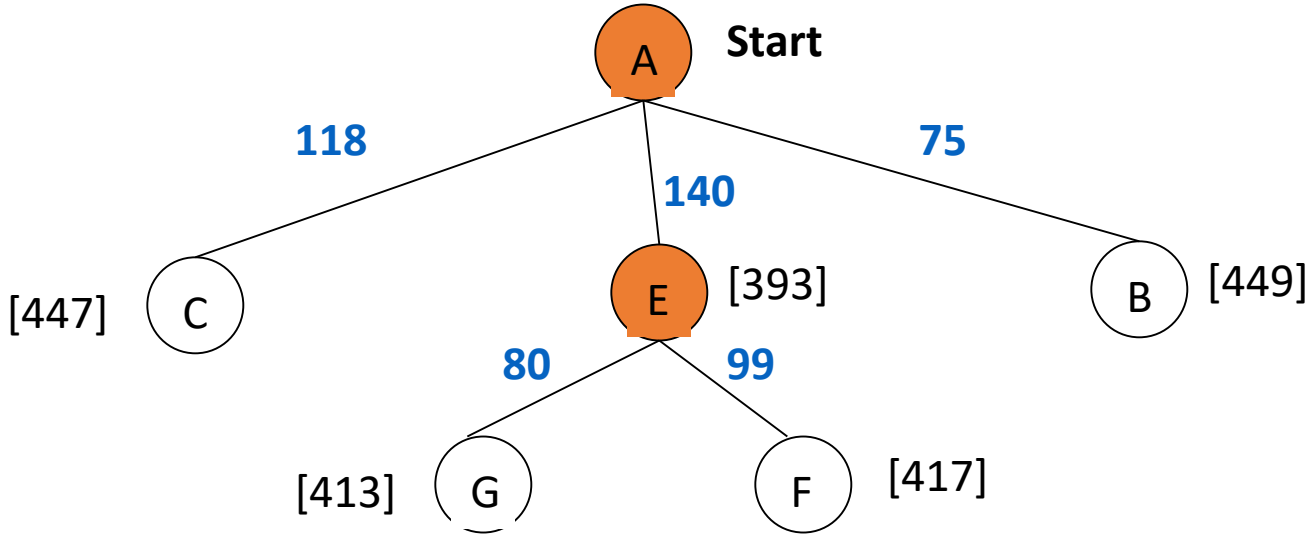
A* Search: Tree Search



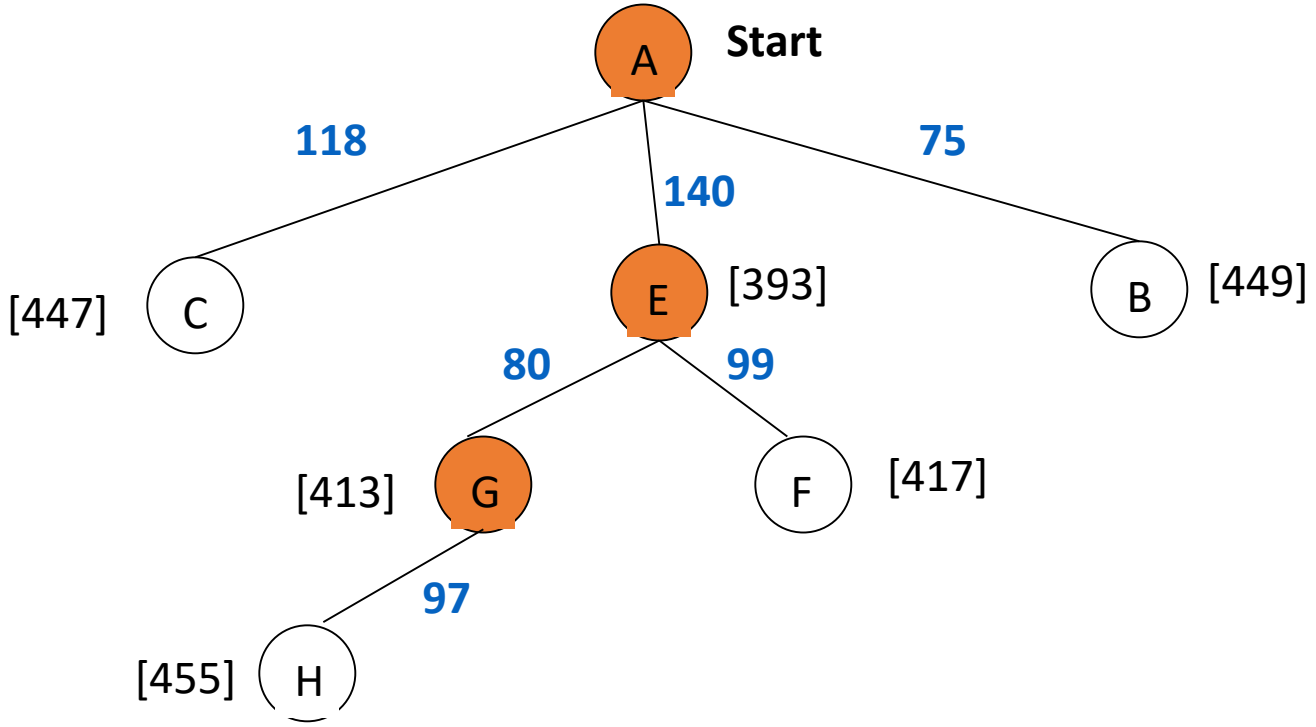
A* Search: Tree Search



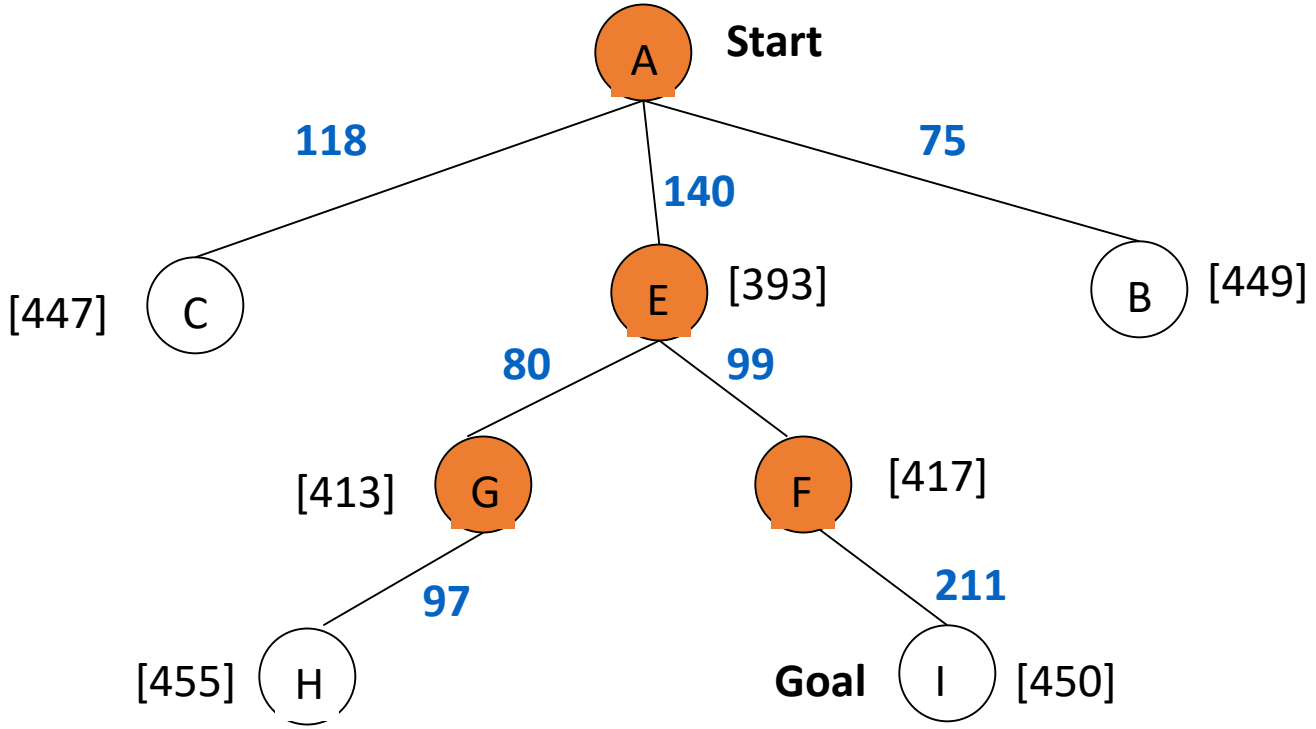
A* Search: Tree Search



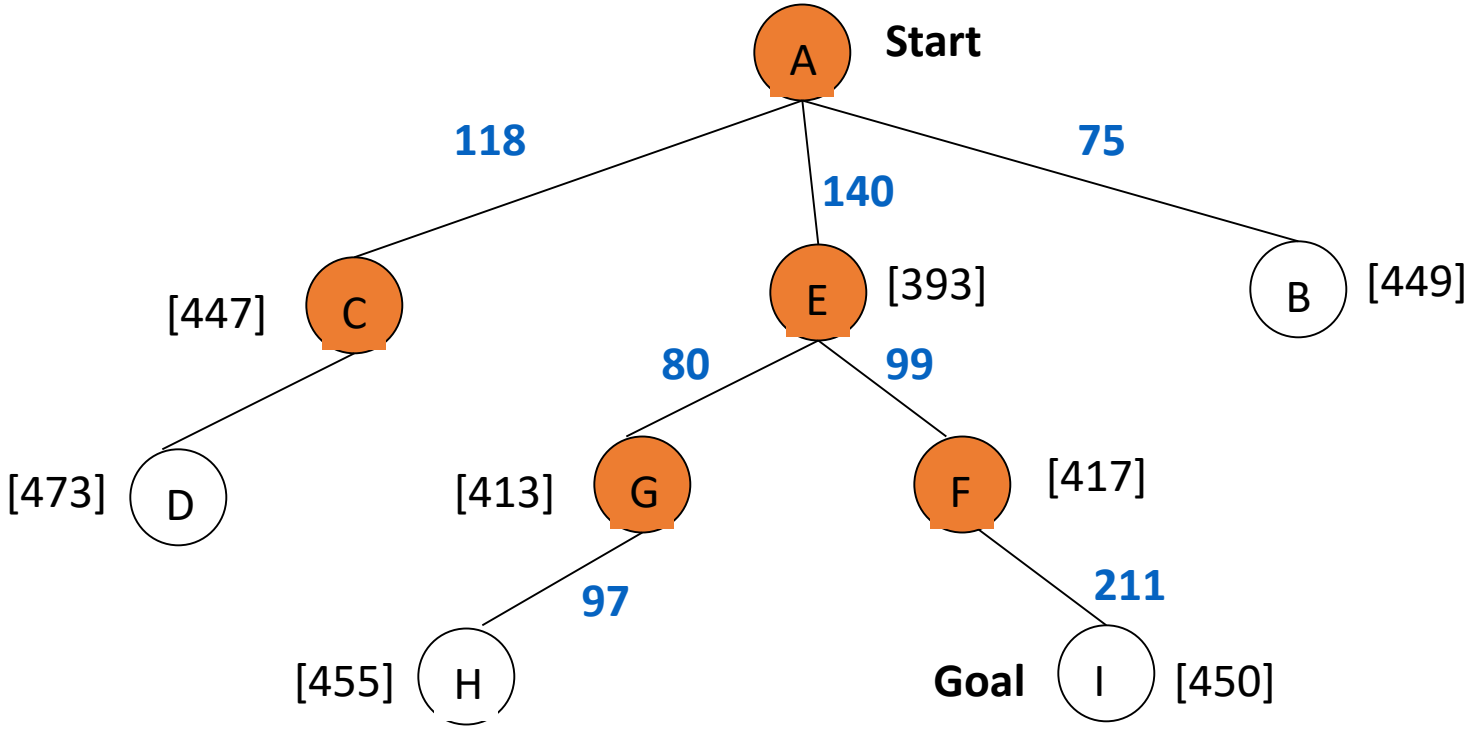
A* Search: Tree Search



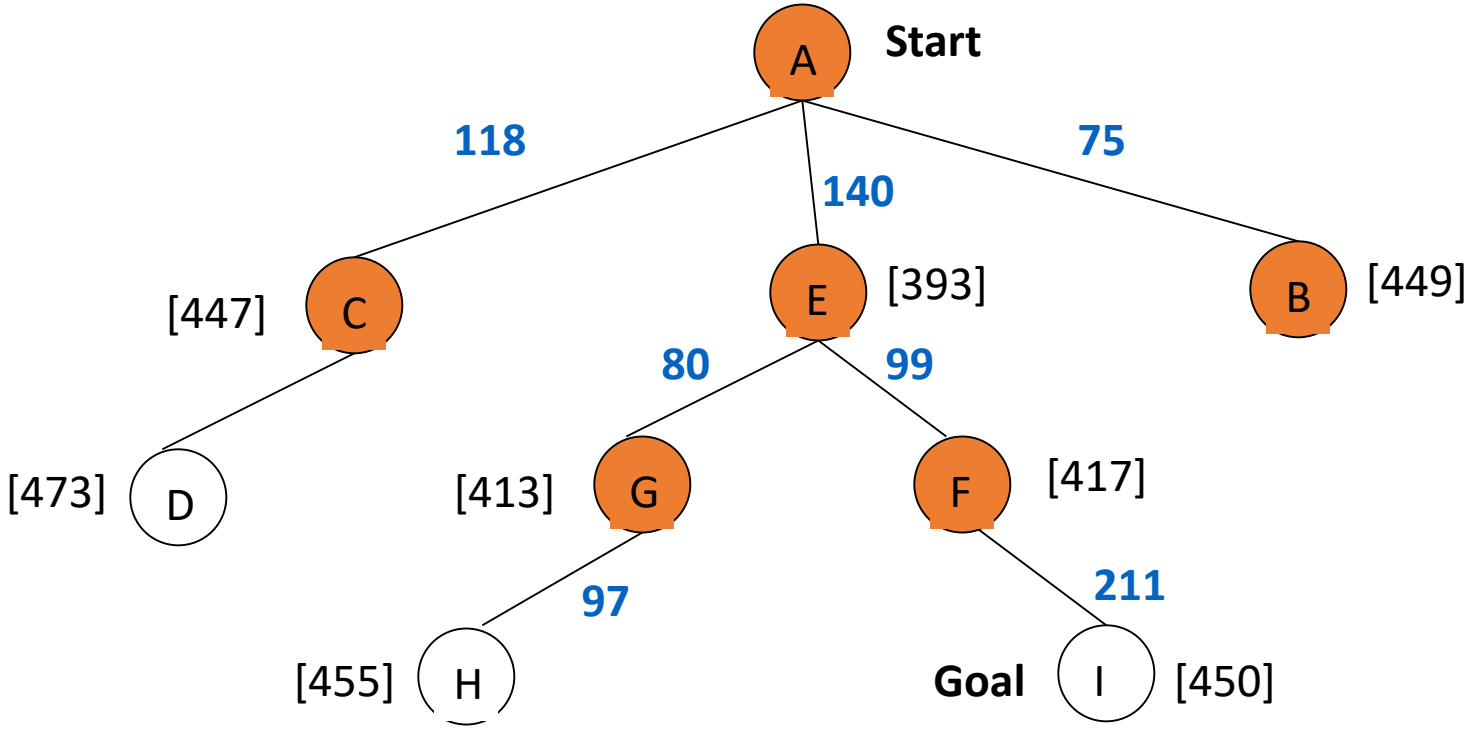
A* Search: Tree Search



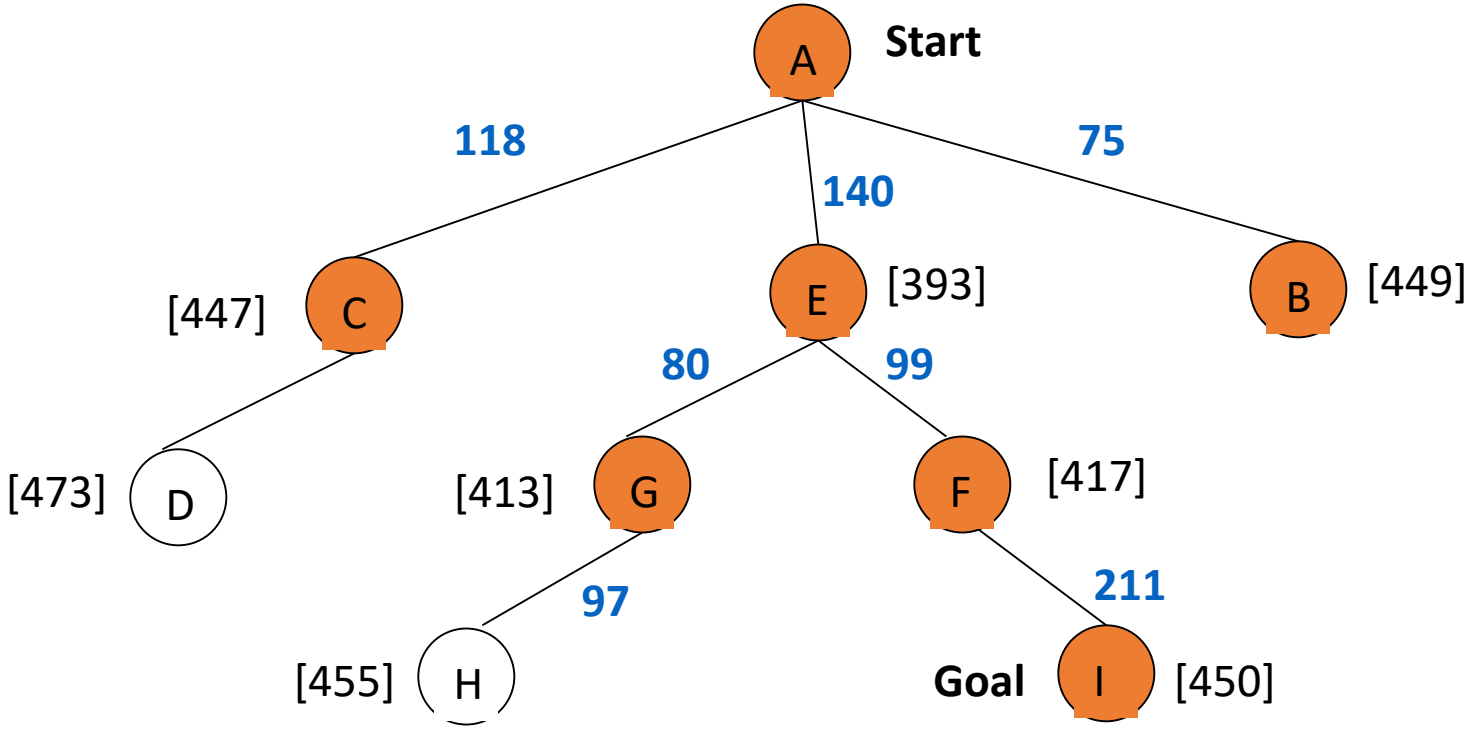
A* Search: Tree Search



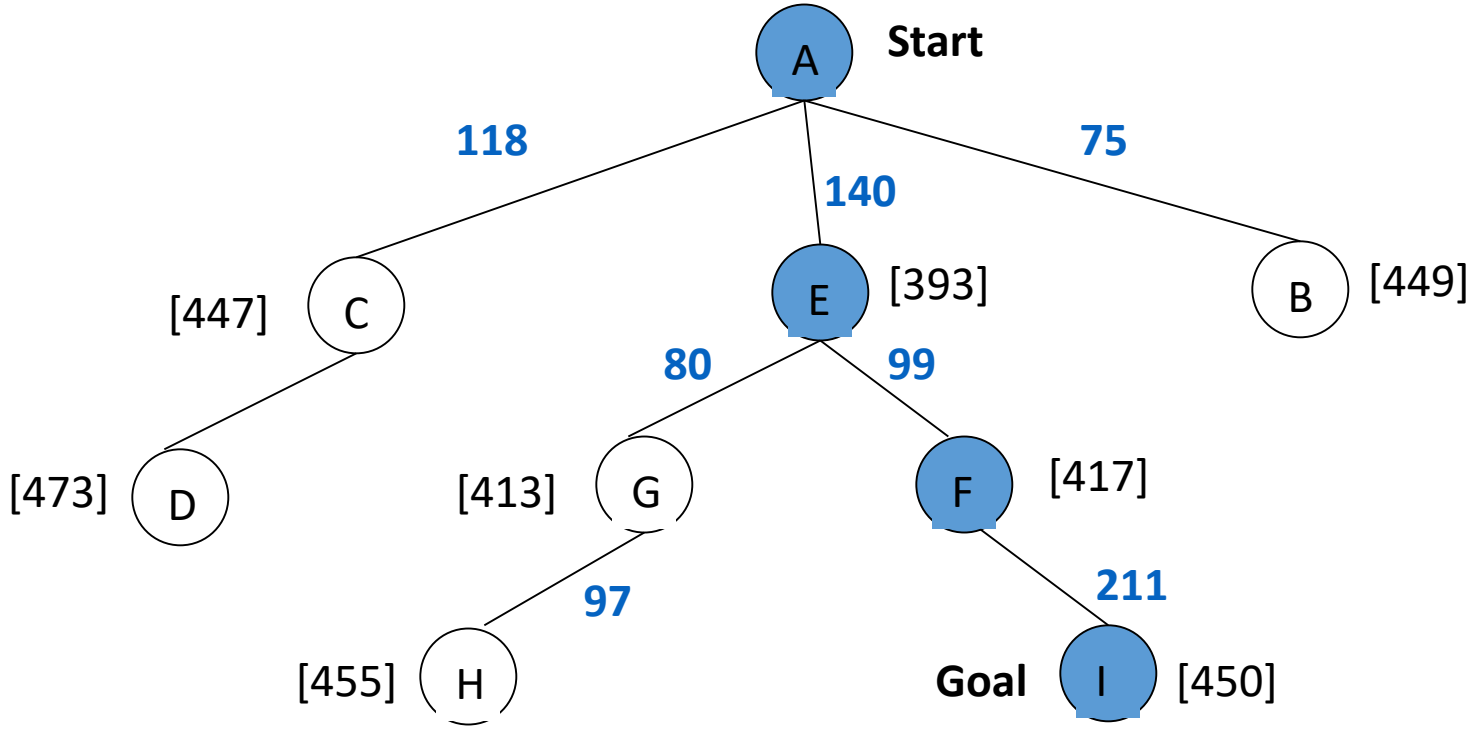
A* Search: Tree Search



A* Search: Tree Search



A* Search: Tree Search



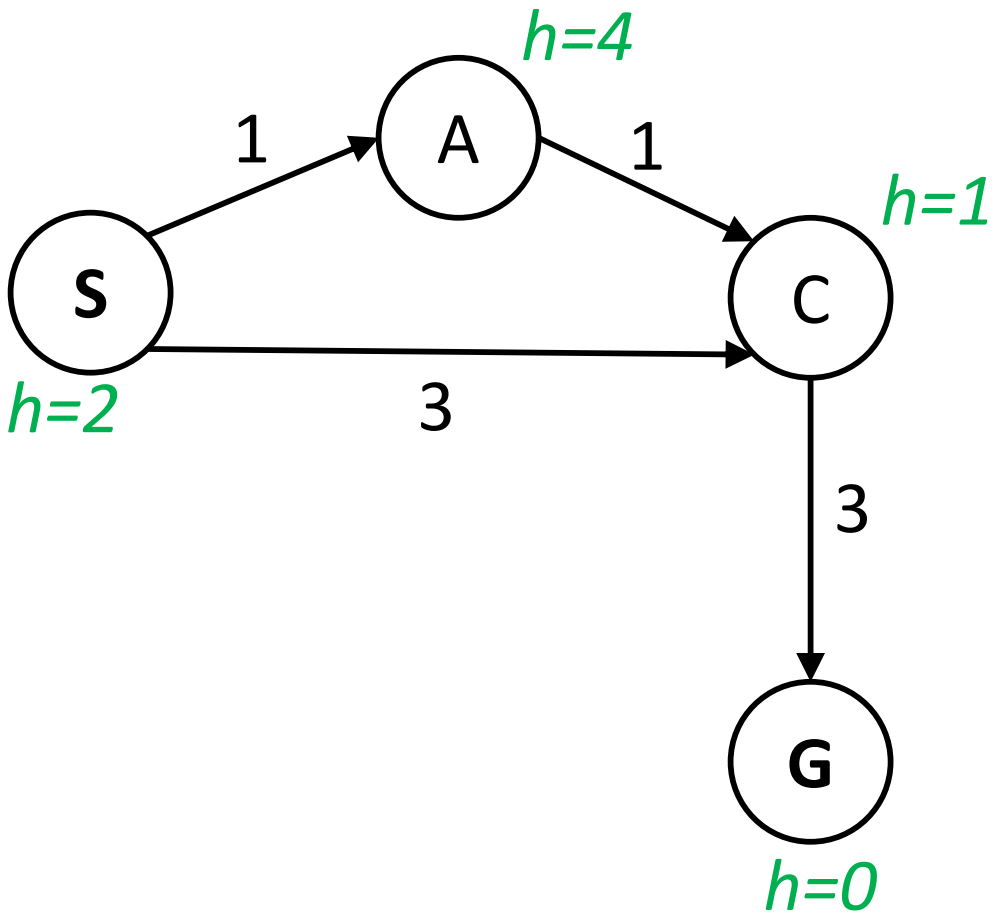
A* not optimal !!!

When heuristic h is not admissible

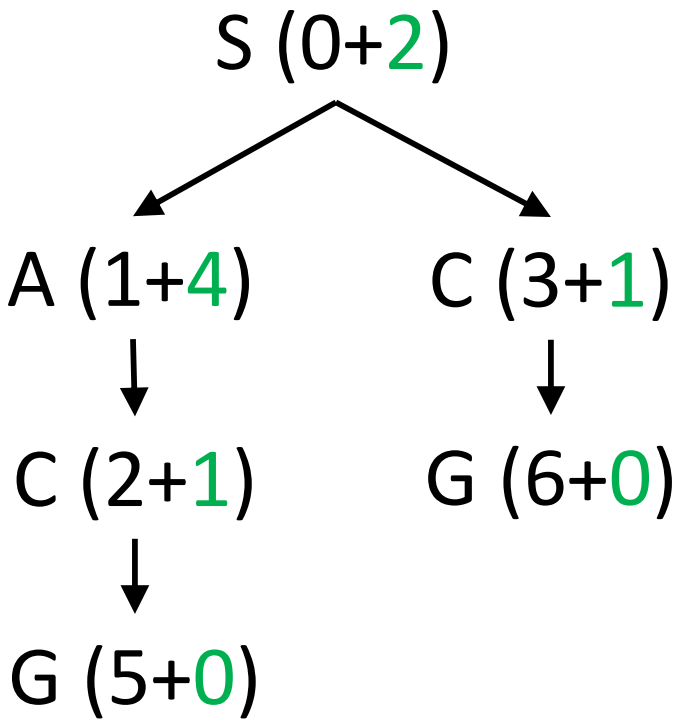
dist(A-E-F-I) =140+99+211=450

A* Tree Search

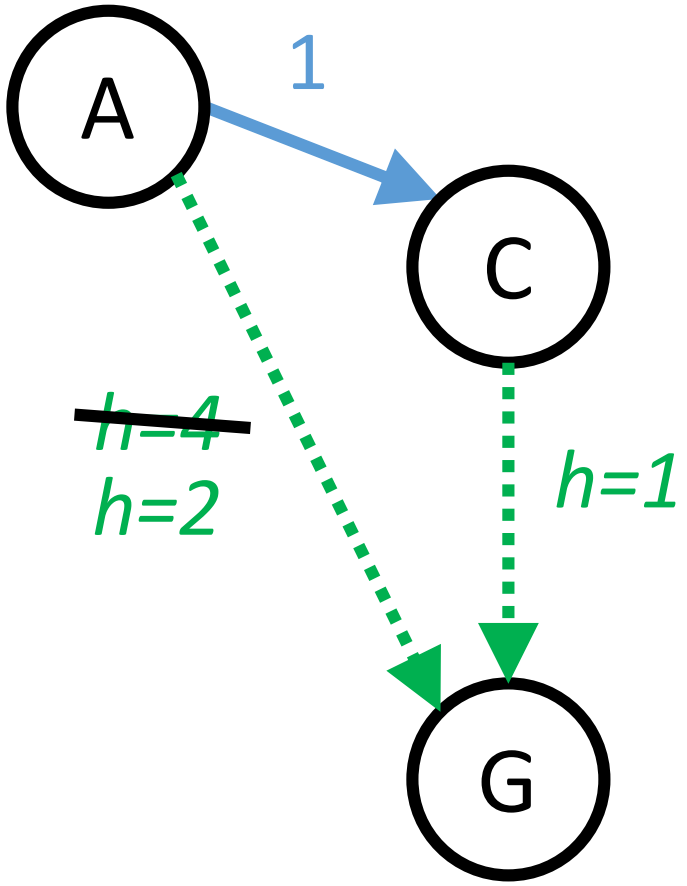
State space graph



Search tree



Consistency of Heuristics



- Main idea: Estimated heuristic costs ≤ actual costs
  - Admissibility:
    heuristic cost ≤ actual cost to goal
    h(A) ≤ actual cost from A to G
  - Consistency:
    "heuristic step cost" ≤ actual cost for each step
    h(A) – h(C) ≤ cost(A to C)
    triangle inequality
    h(A) ≤ cost(A to C) + h(C)
- Consequences of consistency:
  - The f value along a path never decreases
  - A* graph search is optimal

But an admissible heuristic is not always consistent

A* Algorithm

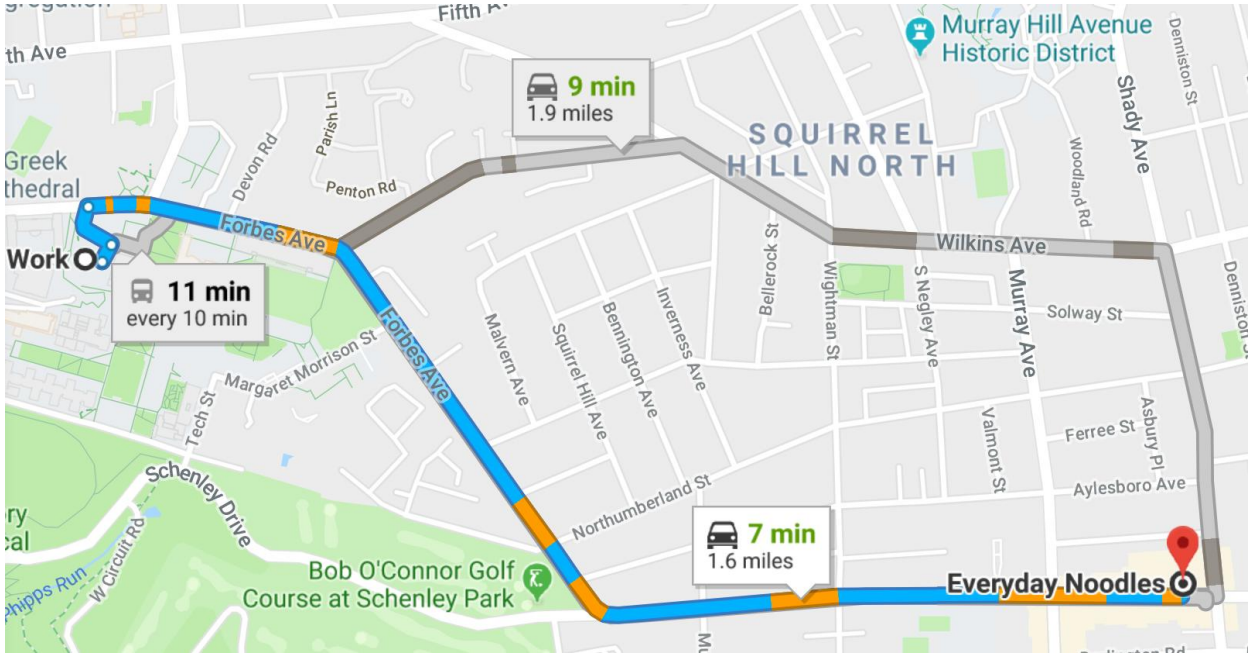
1. Search queue Q is empty.
2. Place the start state s in Q with f value h(s).
3. If Q is empty, return failure.
4. Take node n from Q with lowest f value.
   (Keep Q sorted by f values and pick the first element).
5. If n is a goal node, stop and return solution.
6. Generate successors of node n.
7. For each successor n' of n do:
   a) Compute f(n') = g(n) + h(n').
   b) If n' is new (never generated before), add n' to Q.
   c) If node n' is already in Q with a higher f value, replace it with current f(n') and place it in sorted order in Q.
   End for
8. Go back to step 3.

```
function UNIFORM-COST-SEARCH(problem) returns a solution, or failure
    initialize the explored set to be empty
    initialize the frontier as a priority queue using g(n) as the priority
    add initial state of problem to frontier with priority g(S) = 0
    loop do
        if the frontier is empty then
            return failure
        choose a node and remove it from the frontier
        if the node contains a goal state then
            return the corresponding solution
        add the node state to the explored set
        for each resulting child from node
            if the child state is not already in the frontier or explored set then
                add child to the frontier
            else if the child is already in the frontier with higher g(n) then
                replace that frontier node with child
```

```
function A-STAR-SEARCH(problem) returns a solution, or failure
    initialize the explored set to be empty
    initialize the frontier as a priority queue using f(n) = g(n) + h(n) as the priority
    add initial state of problem to frontier with priority f(S) = 0 + h(S)
    loop do
        if the frontier is empty then
            return failure
        choose a node and remove it from the frontier
        if the node contains a goal state then
            return the corresponding solution
        add the node state to the explored set
        for each resulting child from node
            if the child state is not already in the frontier or explored set then
                add child to the frontier
            else if the child is already in the frontier with higher f(n) then
                replace that frontier node with child
```

A* Applications

- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Video games
- Machine translation
- Speech recognition
- …



Properties of A*

Properties of A*

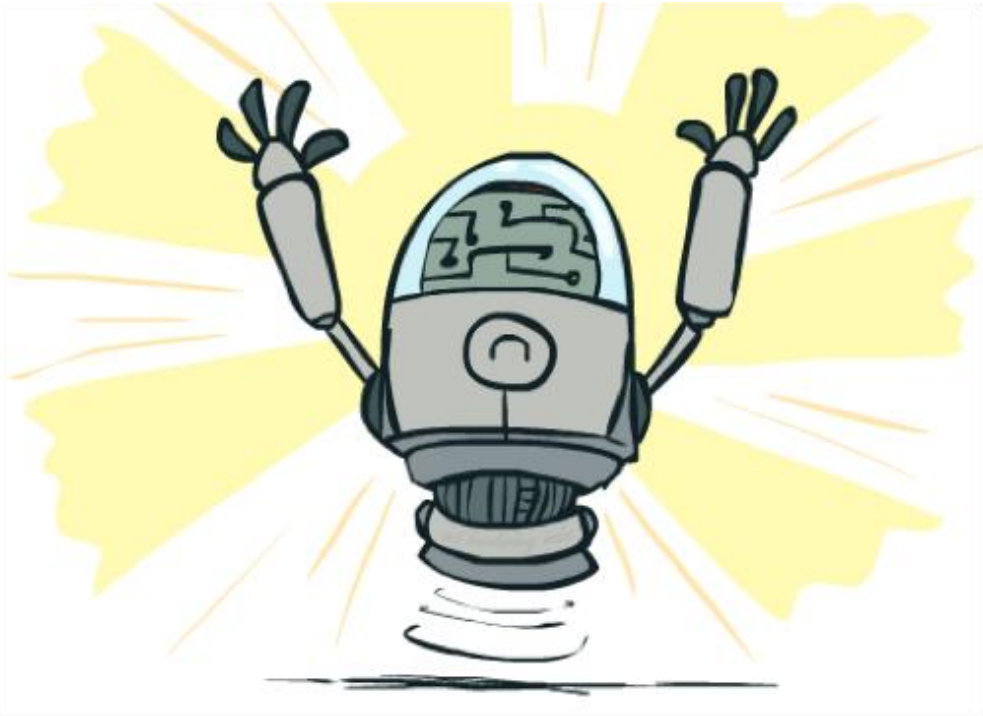
- Complete? **Yes**
  - (unless there are infinitely many nodes with f ≤ f(G))
- Time? **Exponential**
  - It depends on the quality of heuristic but is still exponential.
  - with respect to the length of the solution
  - better than other algorithms, but still problematic
- Space? Keeps all nodes in memory
  - **$O(b^d)$** space complexity,
  - A* keeps all generated nodes in memory
  - But an iterative deepening version is possible (IDA*).
- Optimal? **Yes**
  - A* is optimal if heuristic *h* is admissible (optimal).

Optimality

- Tree search:
  - A* is optimal if heuristic is admissible
  - UCS is a special case (h = 0)

- Graph search:
  - A* optimal if heuristic is consistent
  - UCS optimal (h = 0 is consistent)

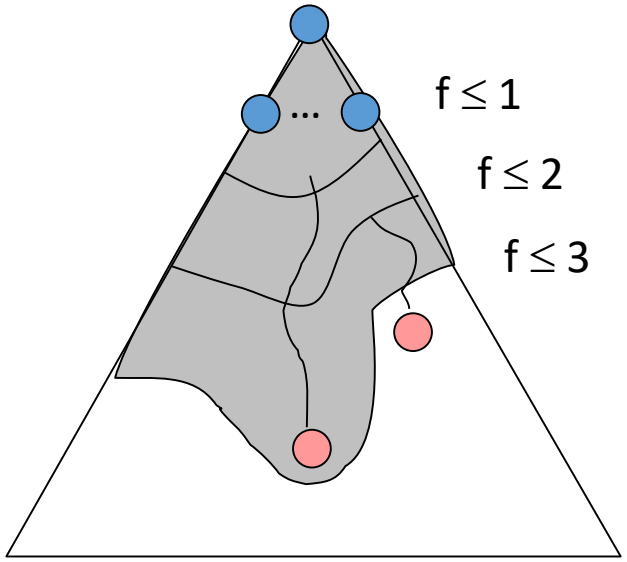
- Consistency implies admissibility



- In general, most natural admissible heuristics tend to be consistent, especially if from relaxed problems

Optimality of A* Graph Search

- Sketch: consider what A* does with a consistent heuristic:
  - Fact 1: In tree search, A* expands nodes in increasing total f value (f-contours)
  - Fact 2: For every state s, nodes that reach s optimally are expanded before nodes that reach s suboptimally



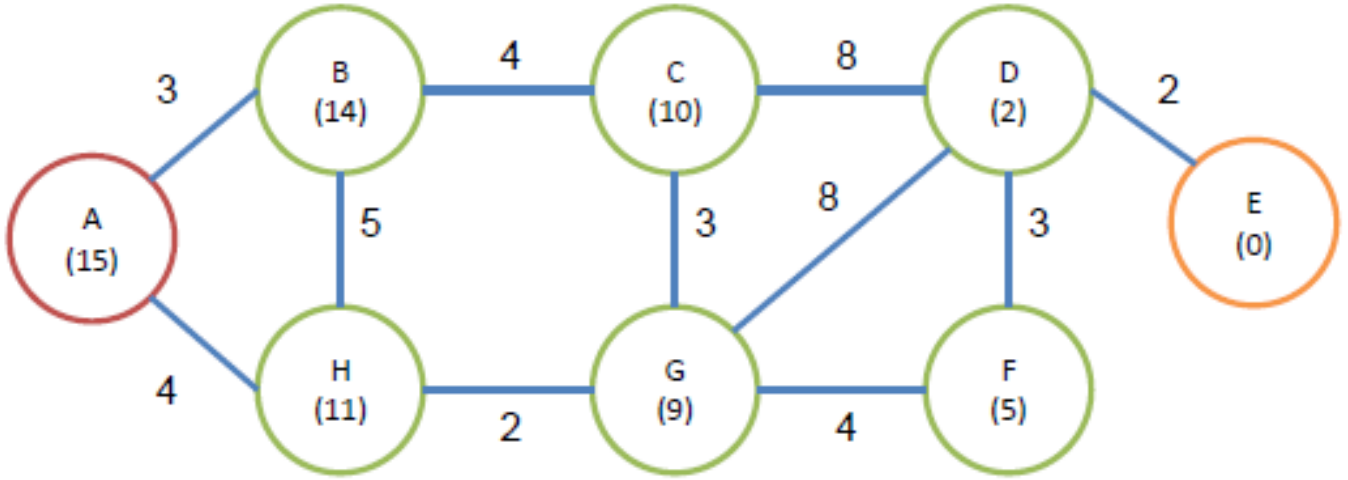
Relationship of Search Algorithms

- Notation
  - g(n) = known cost so far to reach n
  - h(n) = estimated (optimal) cost from n to goal
  - f(n) = g(n)+h(n) = estimated (optimal) total cost through n

- Uniform cost search: sort frontier by g(n)
- Greedy best-first search: sort frontier by h(n)
- A* search: sort frontier by f(n)
  - Optimal for admissible / consistent heuristics
  - Generally the preferred heuristic search framework
  - Memory-efficient versions of A* are available: RBFS, SMA*

Question

- Perform Uniform Cost Search, the Best-First Search, and the A* algorithm. Here we suppose that **A is the initial node, and E is the target**. The cost of each edge and the heuristic value of the each node are also given in the figure.

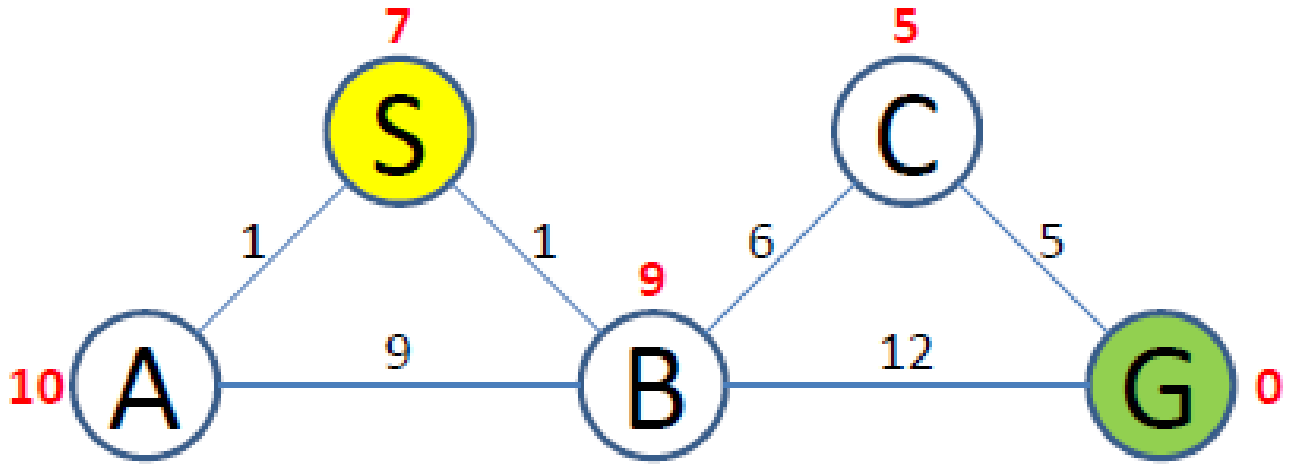


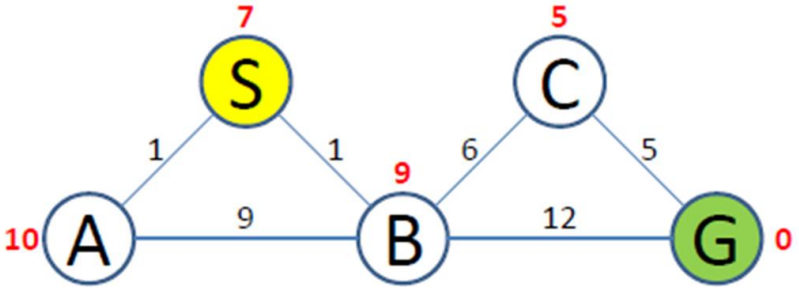
Informed Search

Memory-bounded heuristic search

A* Search

- Perform the A* Algorithm on the following figure. Explicitly write down the queue at each step.

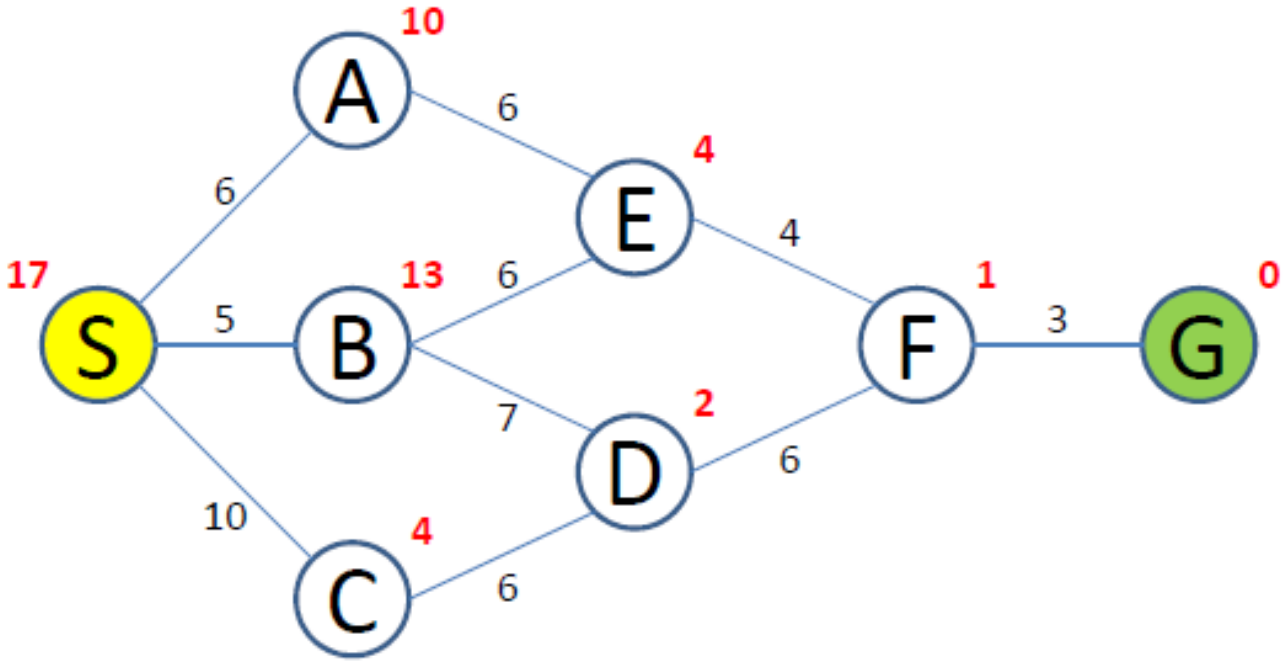


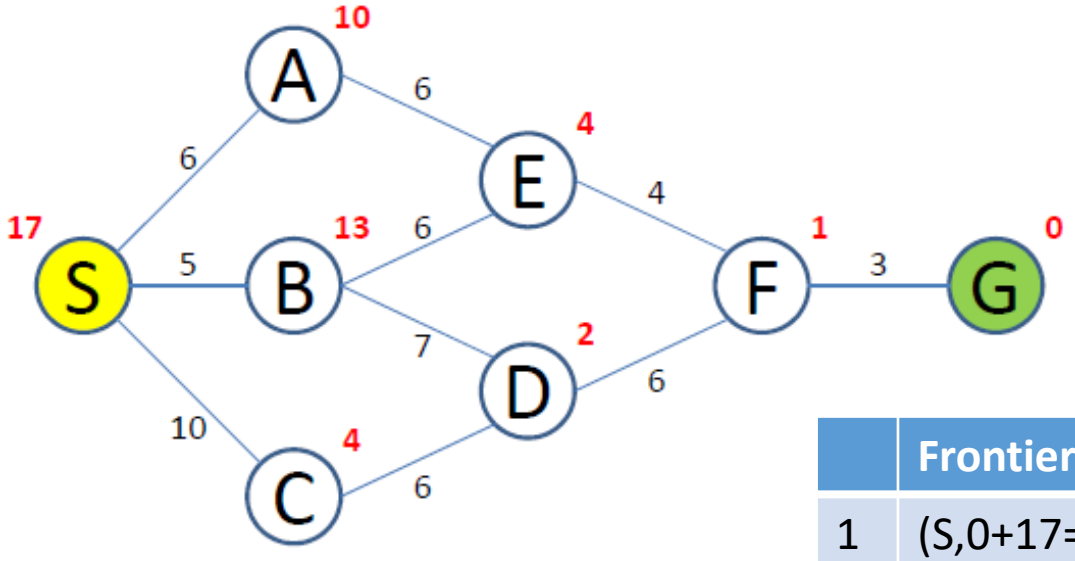


	Frontier (f(n)=g(n)+h(n))	Expand	Explored
1	(S,0+7=7)	S	Empty
2	(S-A,1+10=11)(S-B,1+9=10)	B	S
3	(S-A,11) (S-B-A,{1+9}+10=20) (S-B-C,{1+6}+5=12) (S-B-G,{1+12}+0=13)	A	S,B
4	~~(S-A-B,10+9=19)~~ (S-B-A,{1+9}+10=20) (S-B-C,{1+6}+5=12) (S-B-G,{1+12}+0=13)	C	S,B,A
5	(S-B-A,20) (S-B-C-G,12) (S-B-G,13)	G	S,B,A,C
6	(S-B-C-G,12) Goal Found		S,B,A,C,G

A* Search

- Perform the A* Algorithm on the following figure. Explicitly write down the queue at each step.

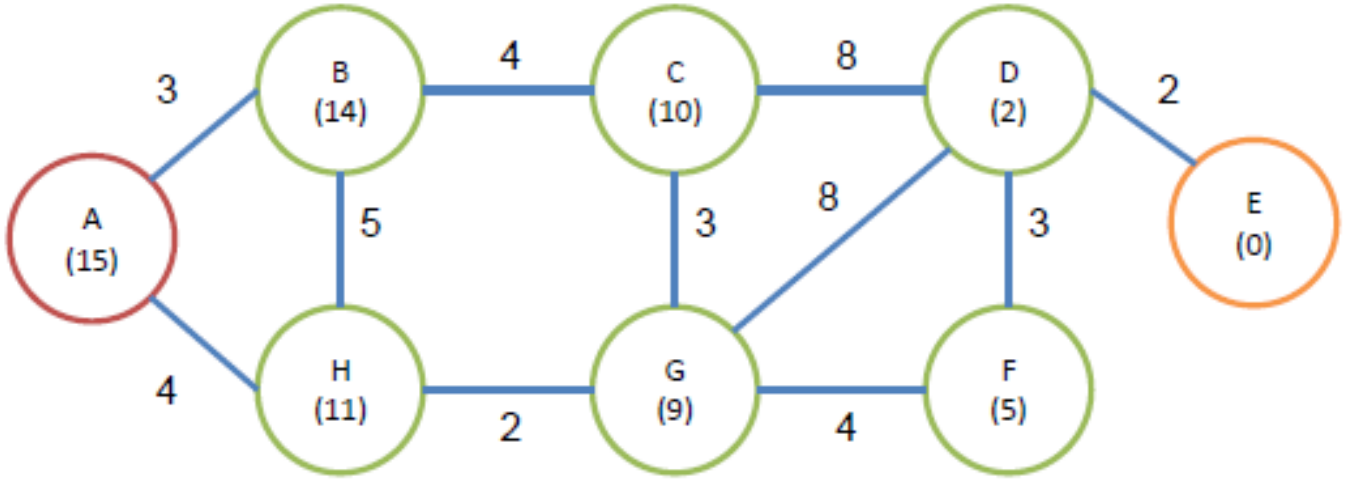


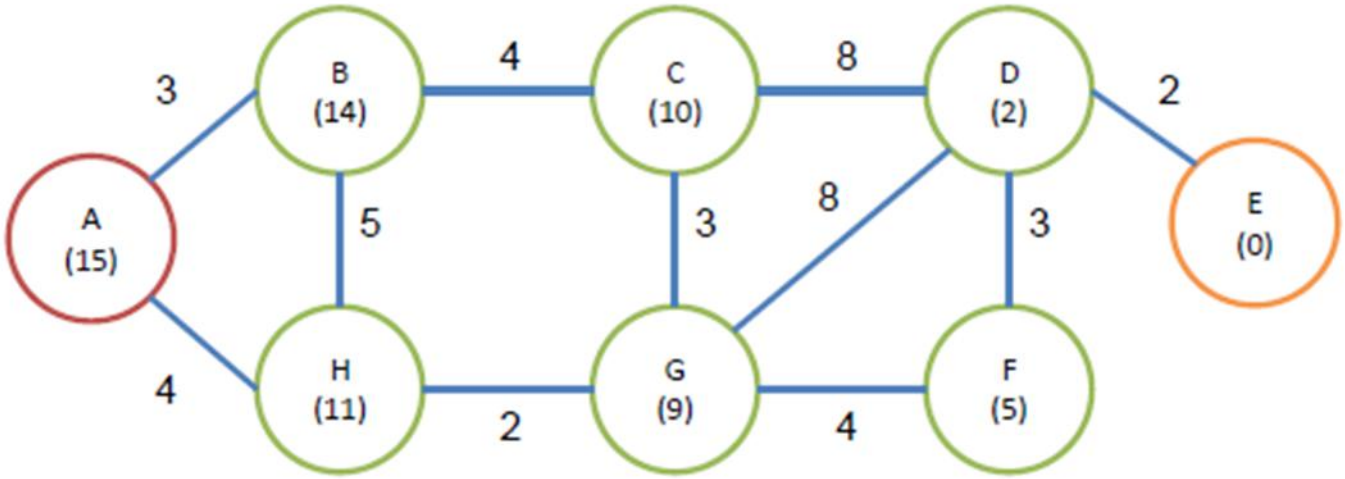


	Frontier	Expand	Explored
1	(S,0+17=17)	S	Empty
2	(S-A,16)(S-B, 18)(S-C,14)	C	S
3	(S-A,16)(S-B, 18)(S-C-D,18)	A	S,C
4	(S-A-E,16)(S-B, 18)(S-C-D,18)	E	S,C,A
5	(S-A-E-B,31)(S-A-E-F,17)(S-B, 18)(S-C-D,18)	F	S,C,A,E
6	(S-A-E-B,31)(S-A-E-F-G,19)(S-B, 18)(S-C-D,18)	B	S,C,A,E,F
7	(S-A-E-F-G,19)(S-B-D,14)(S-C-D,18)	D	S,C,A,E,F,B
8	(S-A-E-F-G,19)~~(S-B-D-F,19)(S-C-D,18)~~	G	
9	(S-A-E-F-G,19) Goal Found		

Question

- Perform Uniform Cost Search, the Best-First Search, and the A* algorithm. Here we suppose that **A is the initial node, and E is the target**. The cost of each edge and the heuristic value of the each node are also given in the figure.

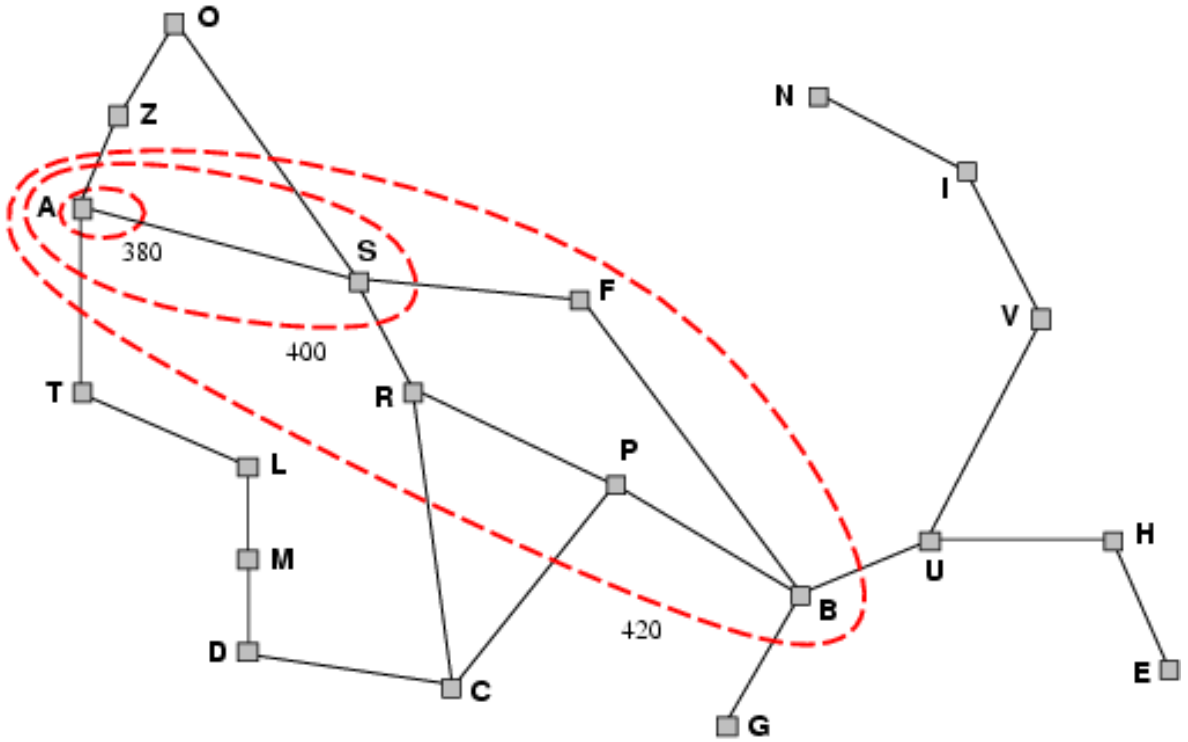




	Frontier	Expand	Explored
1	(A,0+15=15)	A	Empty
2	(A-B,3+14=17)(A-H,4+11=15)	H	A
3	(A-B,17)(A-H-G,6+9=15) (A-H-B,9+14=23)	G	A,H
4	(A-B,17)(A-H-G-F,15) (A-H-G-C,19) (A-H-G-D,16) (A-H-B, 23)	F	A,H,G
5	(A-B,17)(A-H-G-F-D,15) (A-H-G-C,19) (A-H-G-D,16) (A-H-B, 23)	D	A,H,G,F
6	(A-B,17)(A-H-G-F-D-E,15) (A-H-G-F-D-C,31) (A-H-G-C,19) (A-H-G-D,16) (A-H-B, 23)	E	
	(A-H-G-F-D-E,15) Goal Found		

Optimality of A*

- A* expands nodes in order of increasing *f* value
- Gradually adds "*f*-contours" of nodes
- Contour *i* has all nodes with $f=f_i$, where $f_i < f_{i+1}$



Memory-efficient variants of A*

- Some solutions to A* space problems (maintain completeness and optimality)
  - Iterative-deepening A* (IDA*)
    - Here cutoff information is the *f*-cost (*g+h*) instead of depth
  - Recursive best-first search(RBFS)
    - Recursive algorithm that attempts to mimic standard best-first search with linear space.
  - (simple) Memory-bounded A* ((S)MA*)
    - Drop the worst-leaf node when memory is full

Recursive Best-First Search (RBFS)

Recursive Best First Search

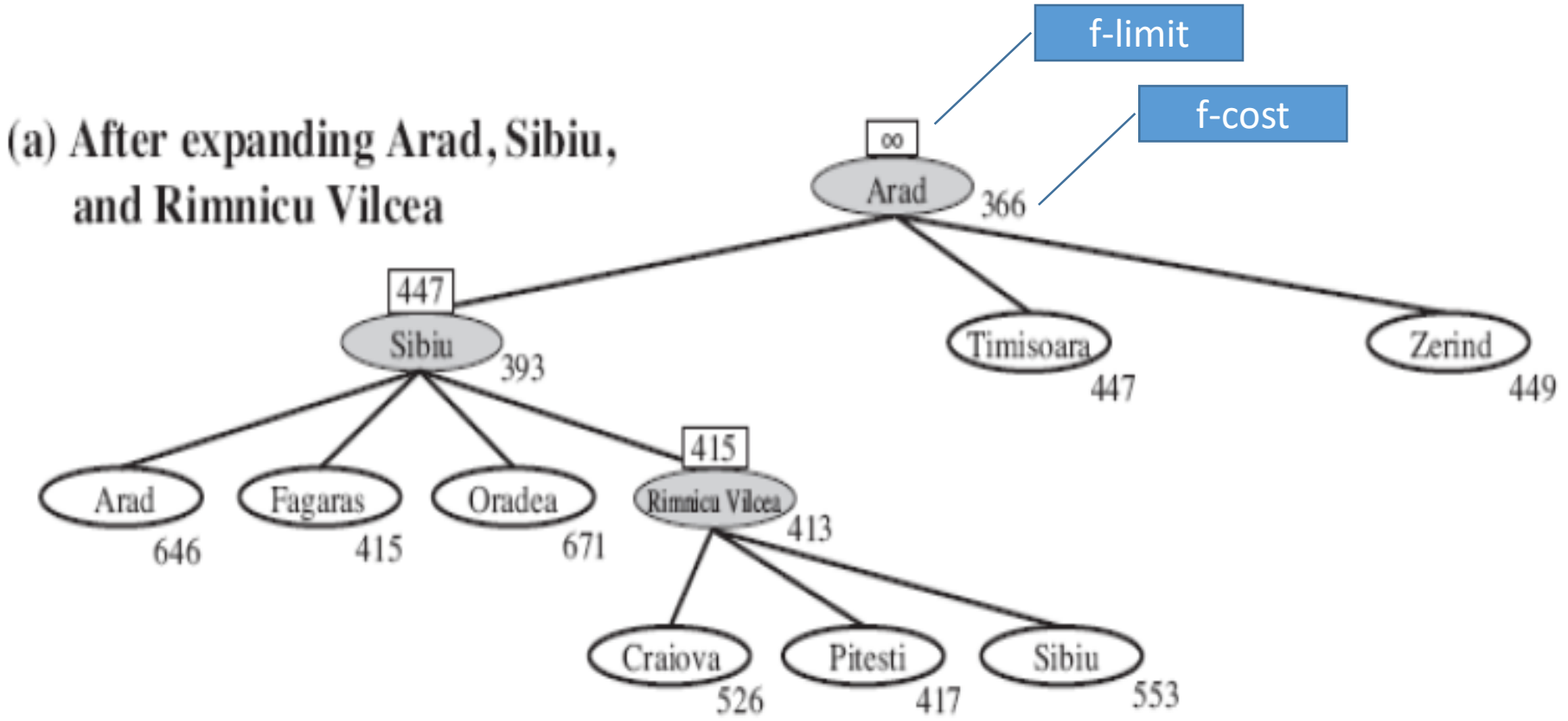
- Uses only linear space and mimics best first search.
- It keeps track of f-value of the best alternative path available from any ancestor of the current node.
- If the current node exceeds this limit, the alternative path is explored.
- RBFS remembers the f-value of the best leaf in the forgotten sub-tree.
- As the recursion unwinds, RBFS replaces the f-value of each node along the path with a **backed-up value**—the best f-value of its children.
- In this way, RBFS remembers the f-value of the best leaf in the forgotten subtree and can therefore decide whether it’s worth re-expanding.

RBFS

- Difference between A* and RBFS
  - A* keeps in memory all of the already generated nodes
  - RBFS only keeps the current search path and the sibling nodes along the path

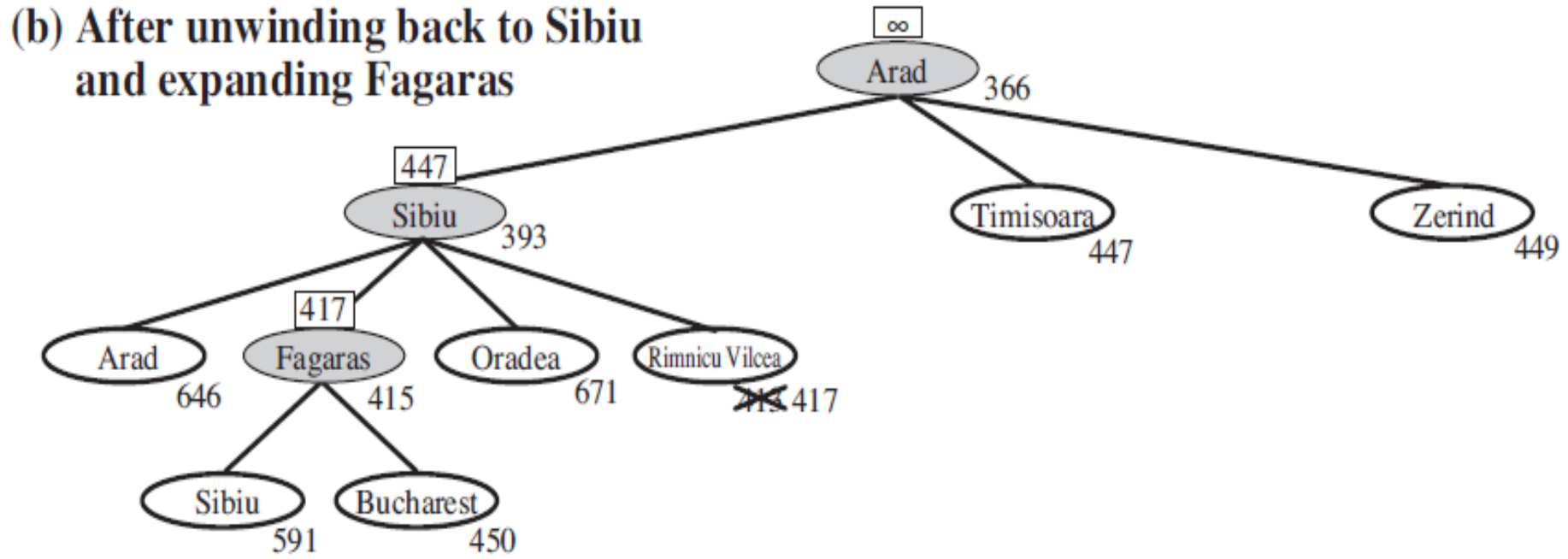
- When does RBFS suspend the search of a subtree?
  - When it no longer looks the best
- What does it do when a subtree is suspended?
  - It forgets the subtree to save space
- What is the space complexity?
  - Linear the depth of the search
- How does RBFS update the f-values?
  - Backing up the f-values in the same way as A* does
- How is the bound determined?
  - The smallest F-value – The closest competitor

RBFS



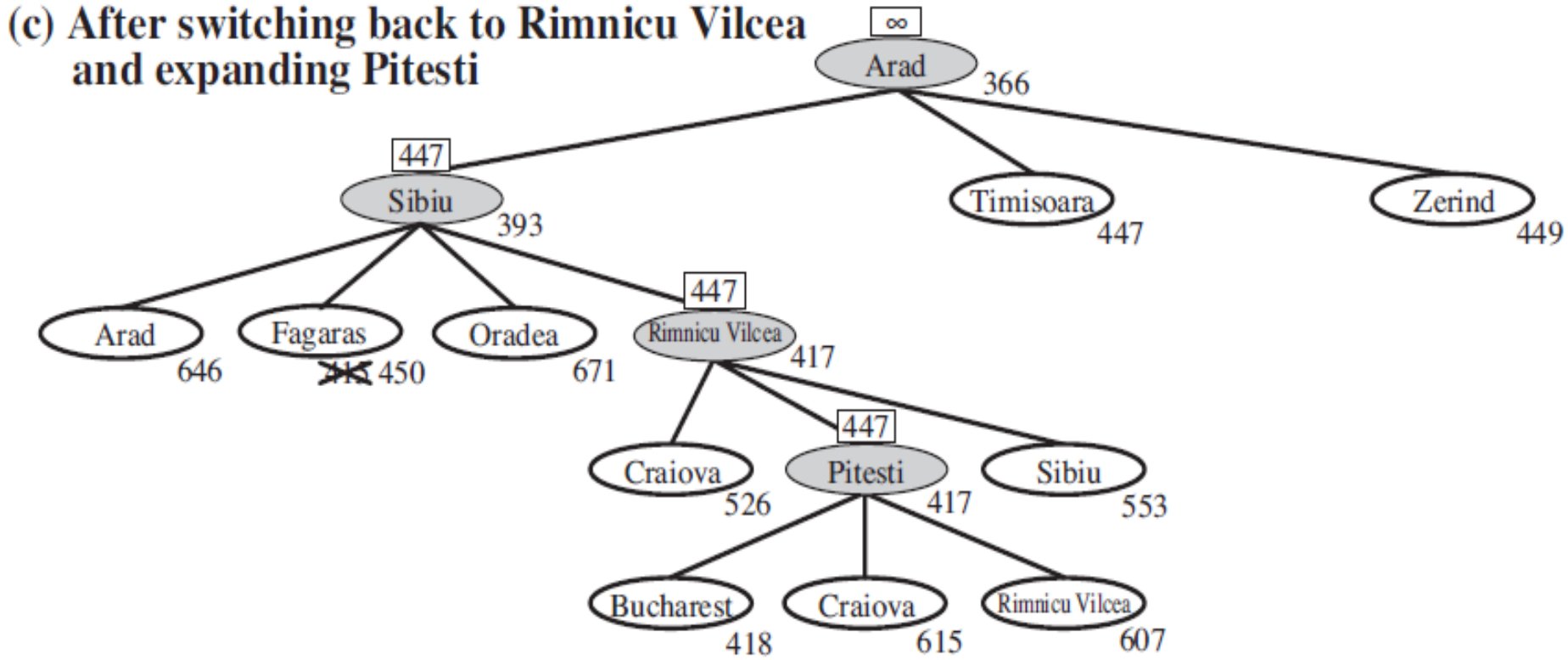
(a) The path via Rimnicu Vilcea is followed until the current best leaf (Pitesti) has a value that is worse than the best alternative path (Fagaras).

RBFS



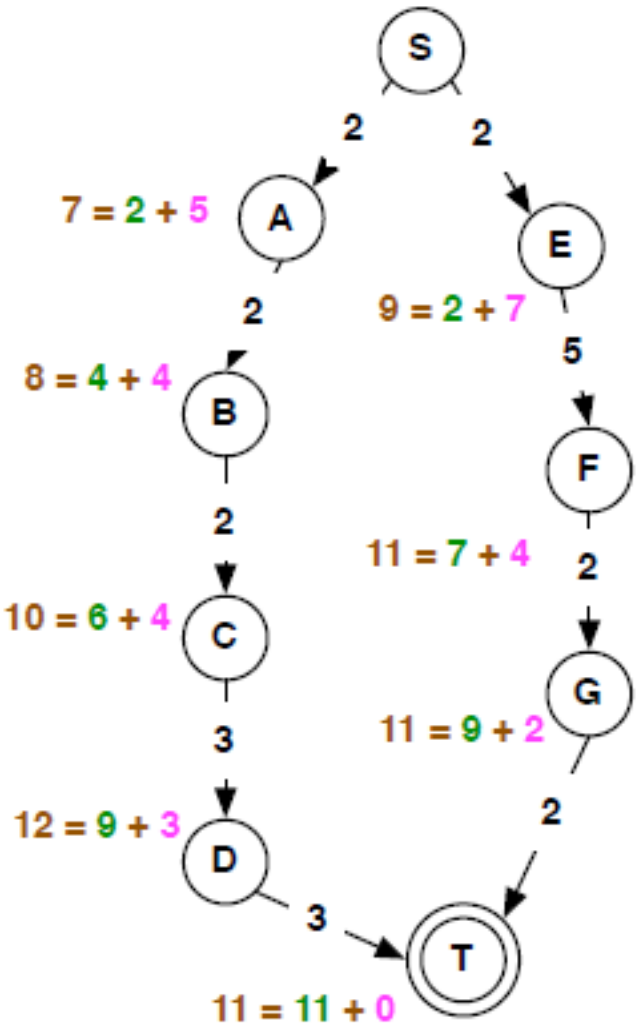
(b) The recursion unwinds and the best leaf value of the forgotten subtree (417) is backed up to Rimnicu Vilcea;
then Fagaras is expanded, revealing a best leaf value of 450.

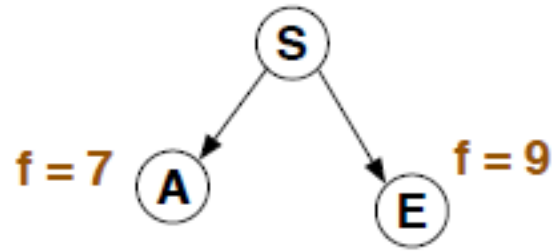
RBFS



(c) The recursion unwinds and the best leaf value of the forgotten subtree (450) is backed up to Fagaras; then Rimnicu Vilcea is expanded. This time, because the best alternative path (through Timisoara) costs at least 447, the expansion continues to Bucharest.

RBFS – 1



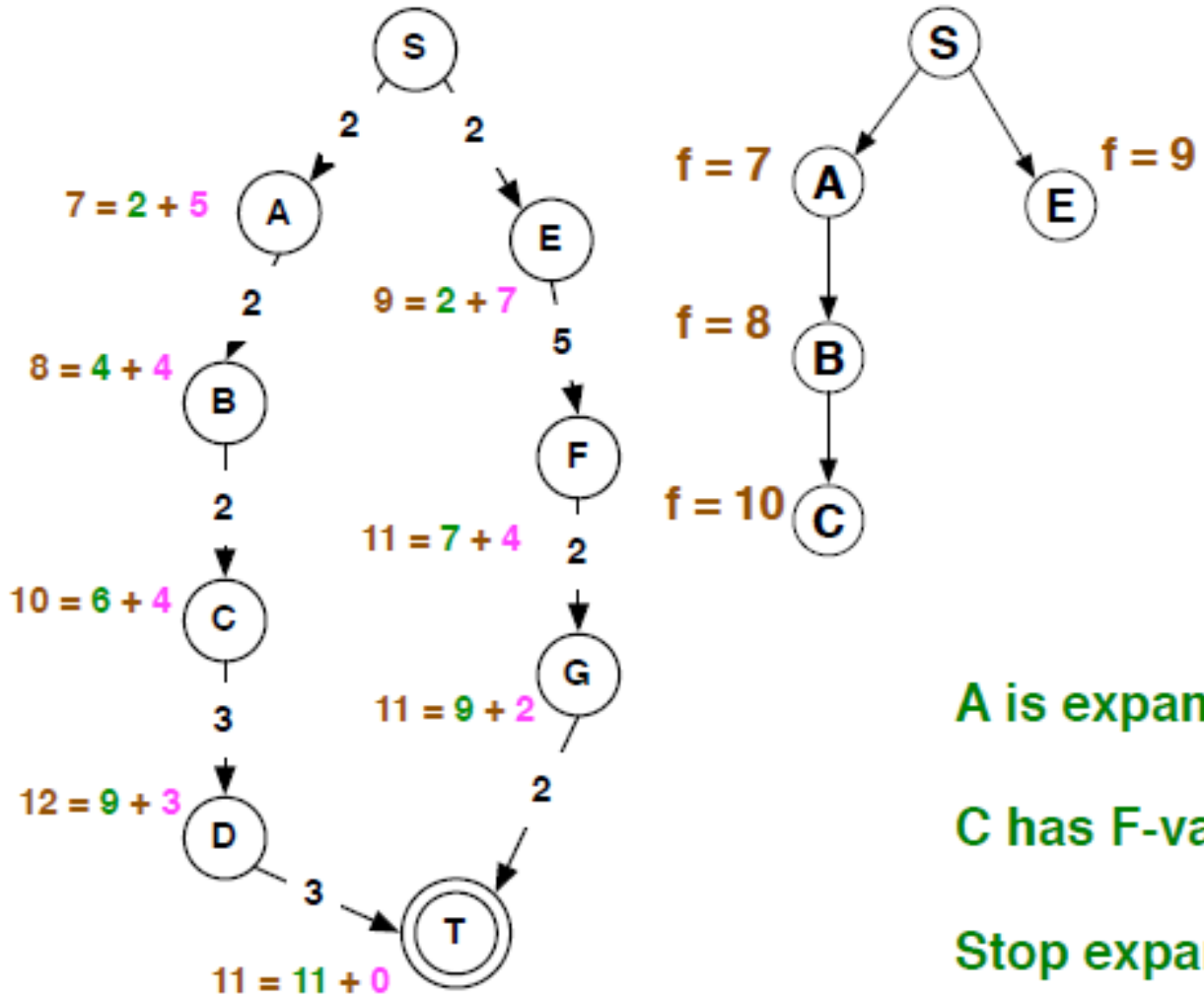


S is expanded

A is found to be the best child

f(n) In mocha = g(n) In clover + h(n) In magenta

RBFS – 2



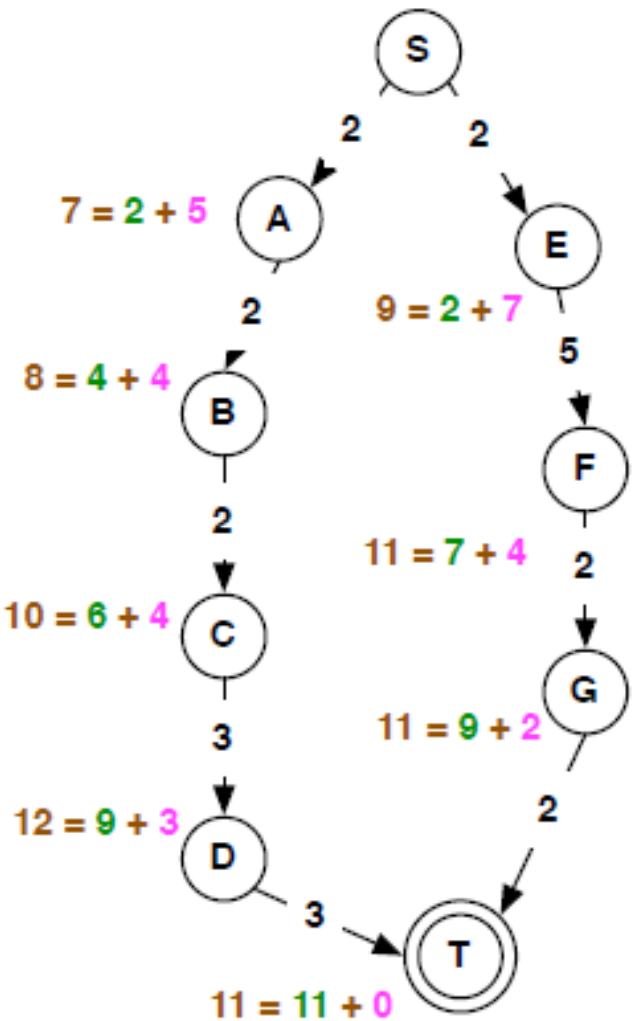
A is expanded with bound 9

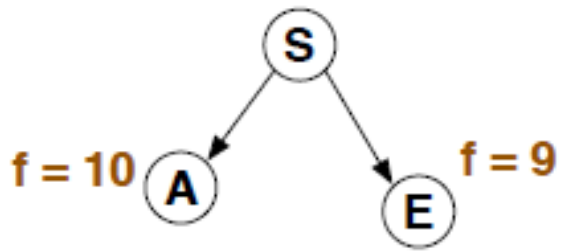
C has F-value 10

Stop expansion, backup F value

f(n) In mocha = g(n) In clover + h(n) In magenta

RBFS – 3





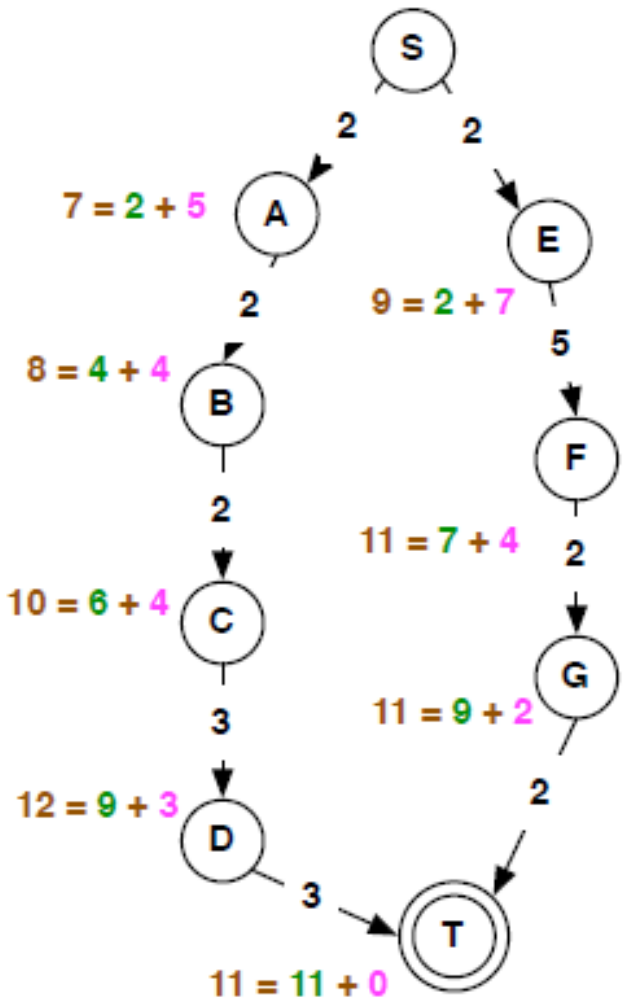
Forget expansion from A

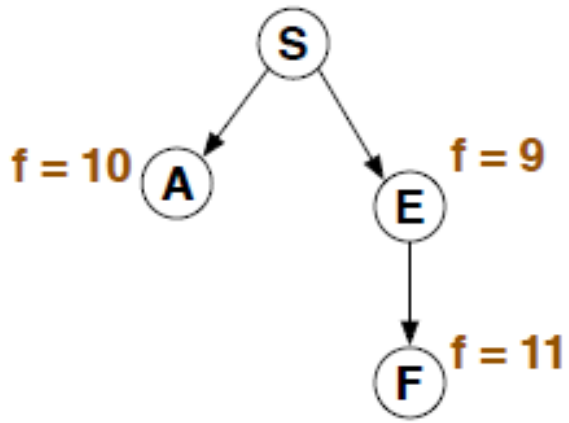
A has backed up F value 10

E is best to expand next

f(n) In mocha = g(n) In clover + h(n) In magenta

RBFS – 4





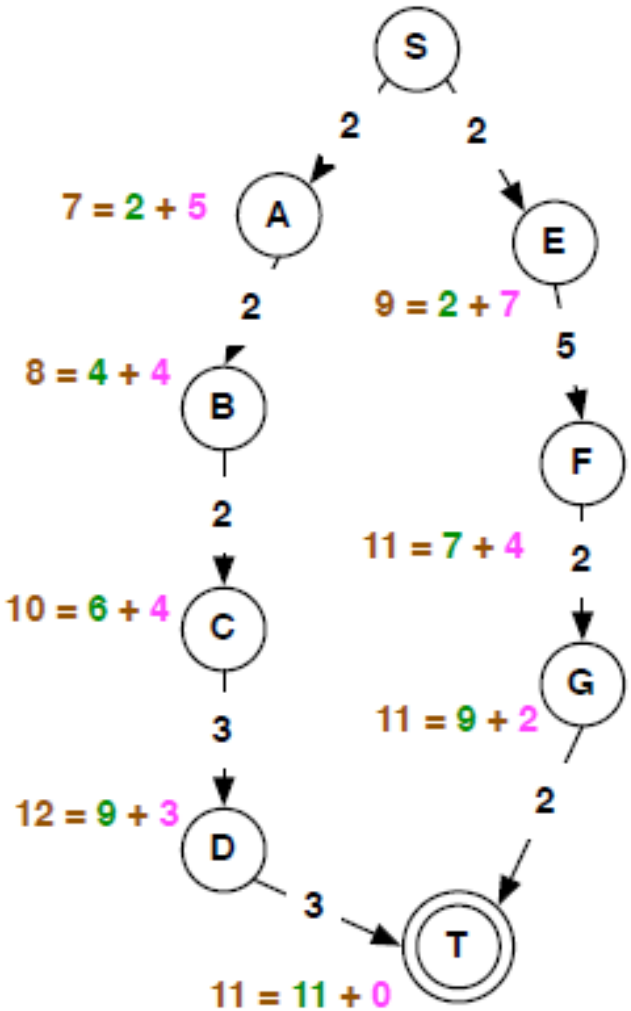
E is expanded with bound 10

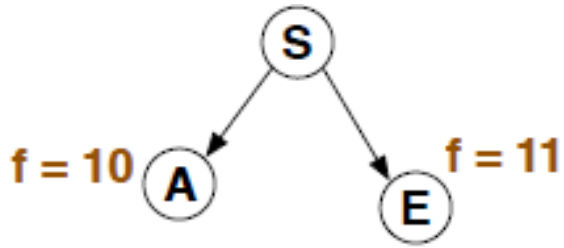
F has F-value 11

Stop expansion, backup F value

f(n) In mocha = g(n) In clover + h(n) In magenta

RBFS – 5



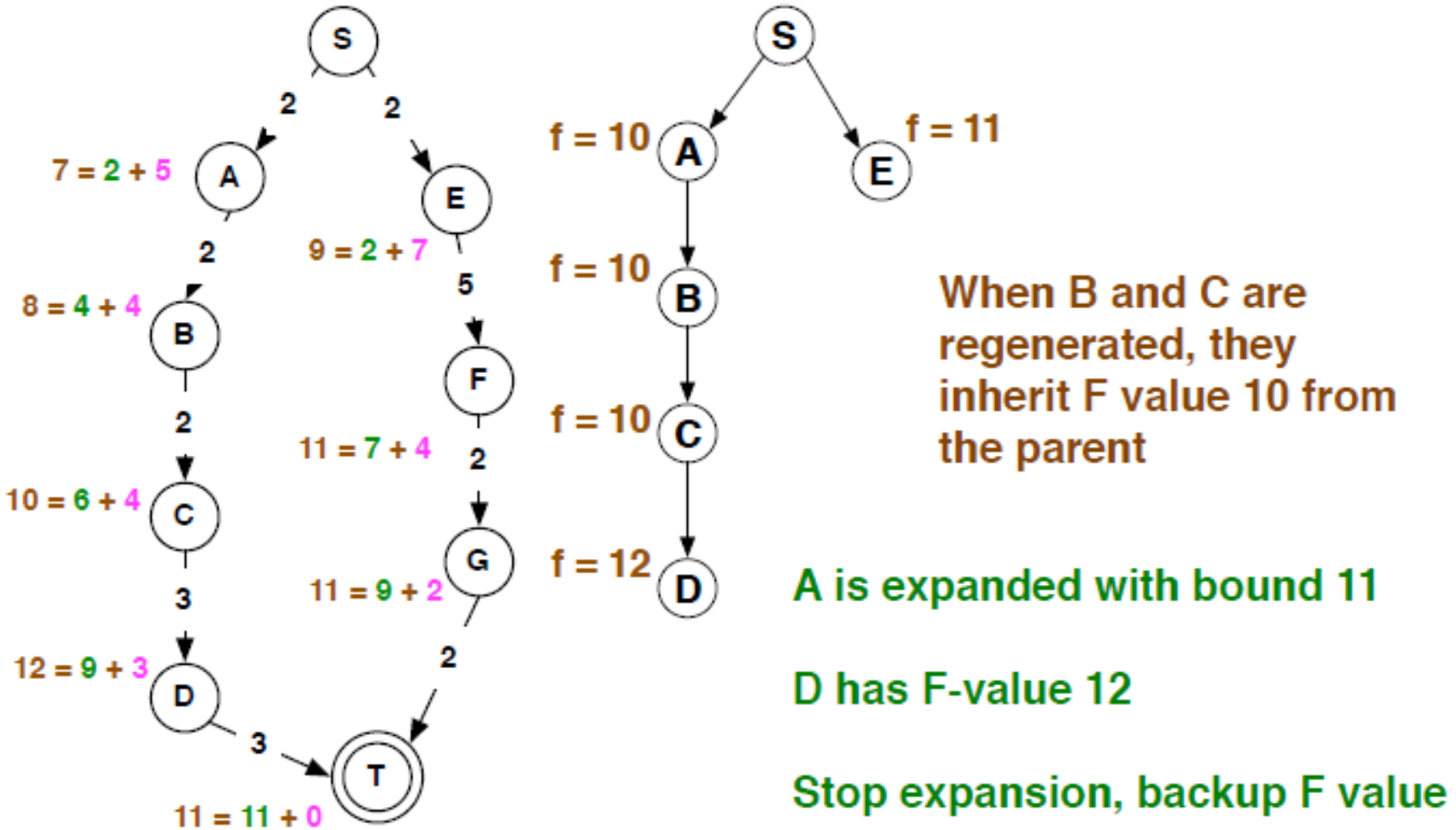


Forget expansion from E

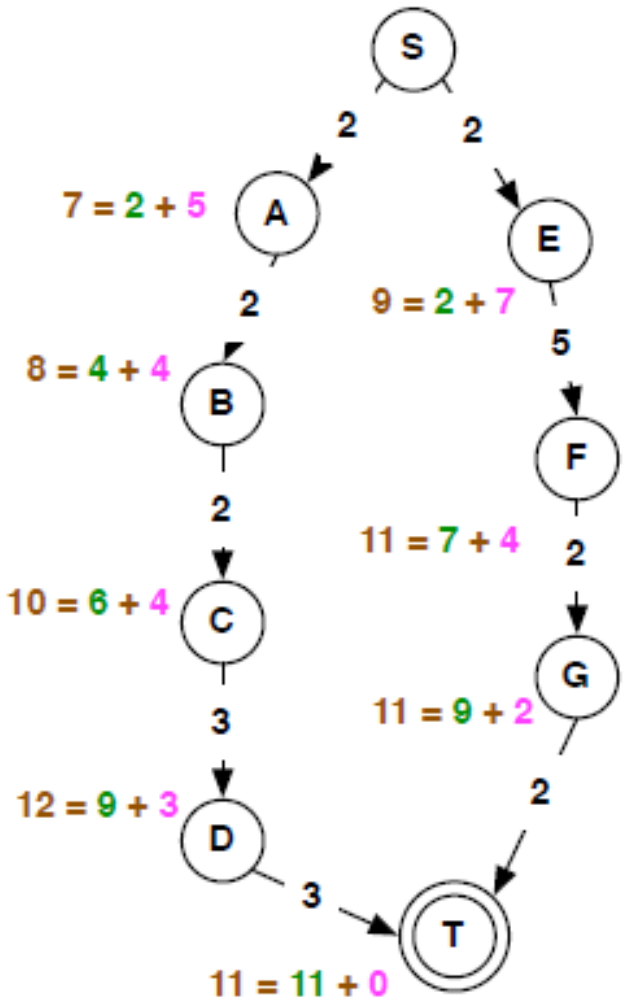
E has backed up F value 11

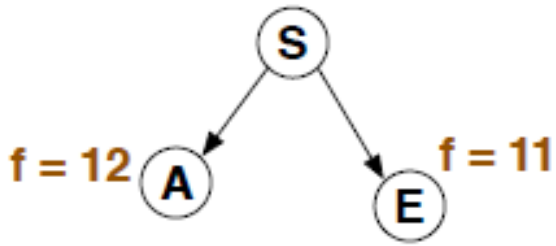
A is best to expand next

RBFS – 6


S
2
2
7 = 2 + 5
A
E
9 = 2 + 7
2
5
8 = 4 + 4
B
F
2
11 = 7 + 4
2
10 = 6 + 4
C
G
3
11 = 9 + 2
2
12 = 9 + 3
D
3
T
11 = 11 + 0
S
f = 10
A
E
f = 11
f = 10
B
f = 10
C
f = 12
D
When B and C are regenerated, they inherit F value 10 from the parent
A is expanded with bound 11
D has F-value 12
Stop expansion, backup F value

RBFS – 7



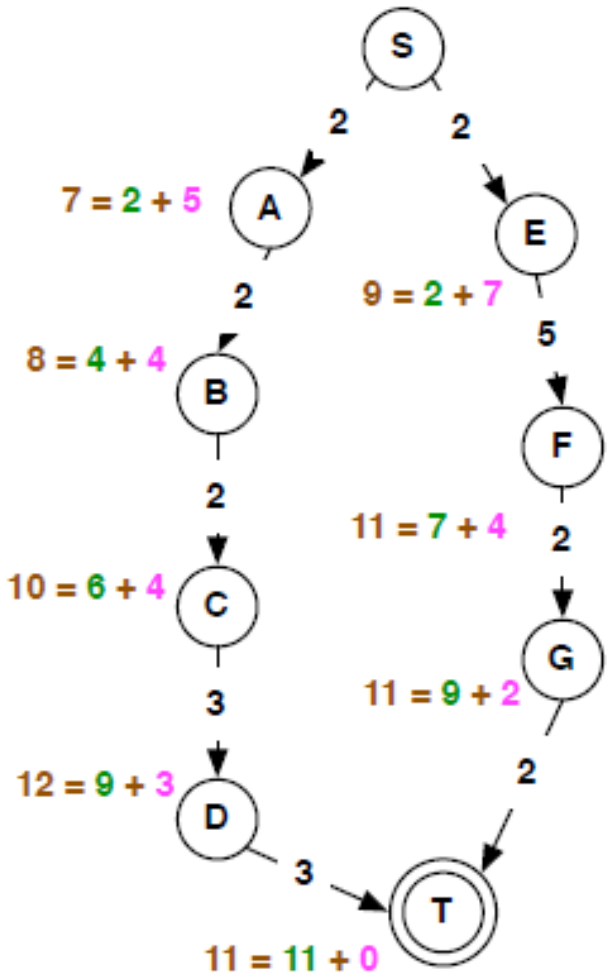


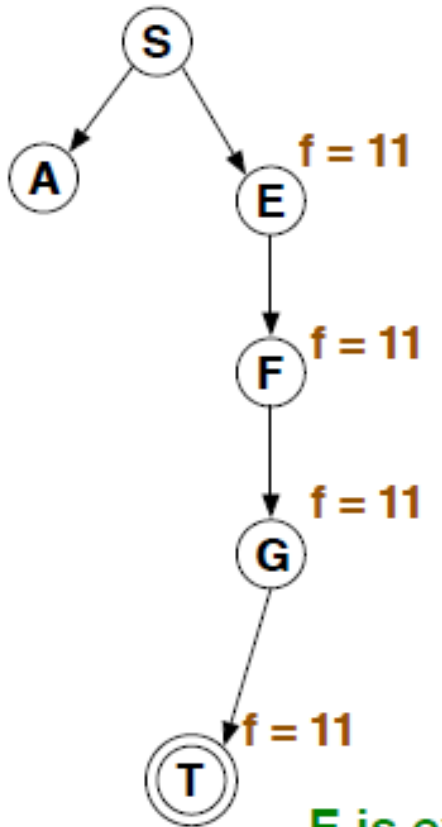
Forget expansion from A

A has backed up F value 12

E is best to expand next

RBFS – 8





E is expanded with bound 12

Reach goal, search ends

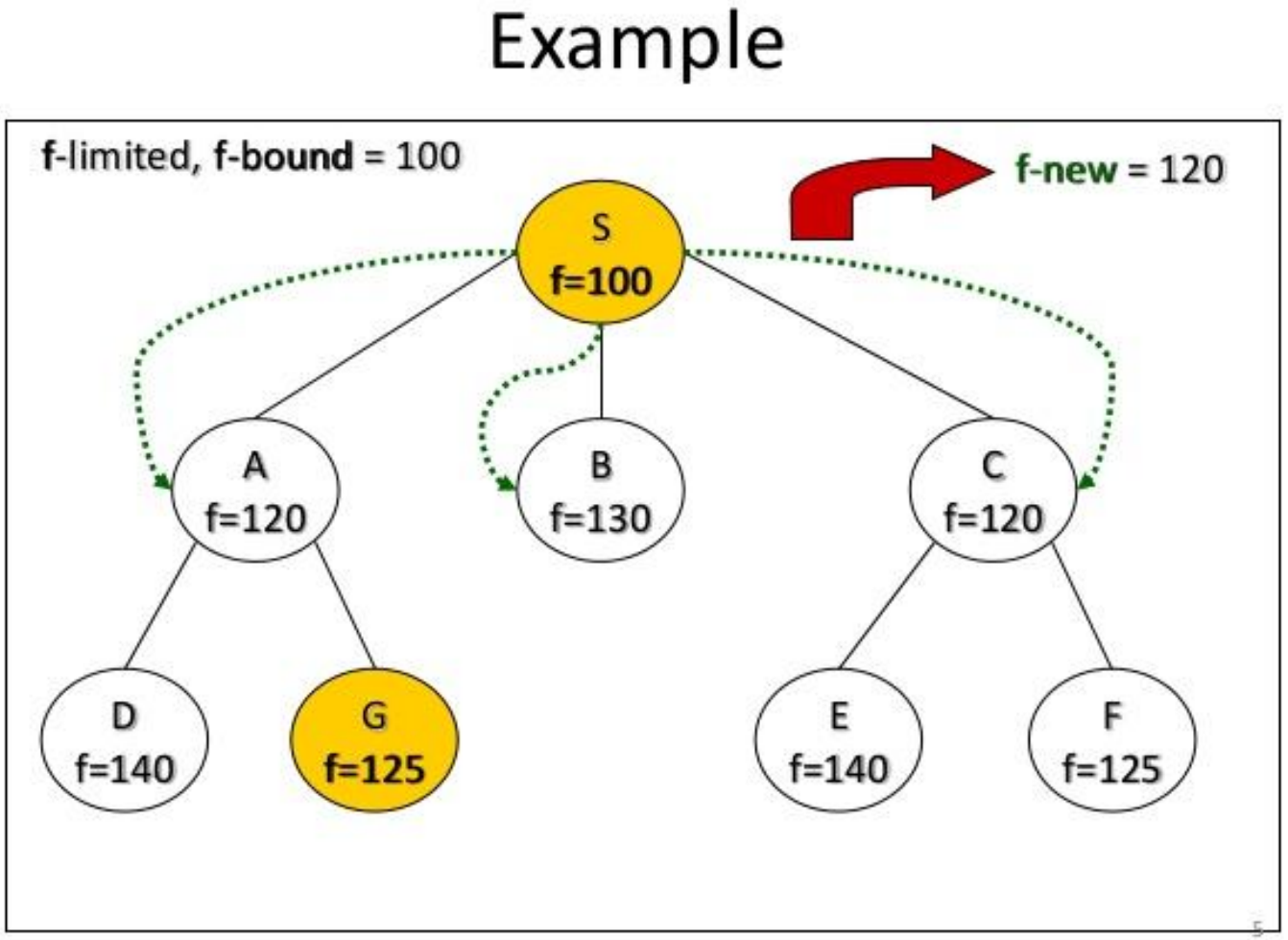
RBFS

- still suffers from excessive node regeneration.
- Optimal
- Its space complexity is linear in the depth of the deepest optimal solution,
- Its time complexity is rather difficult to characterize: depends on the accuracy of the heuristic function and how the best path changes as nodes are expanded.

RBFS – Properties

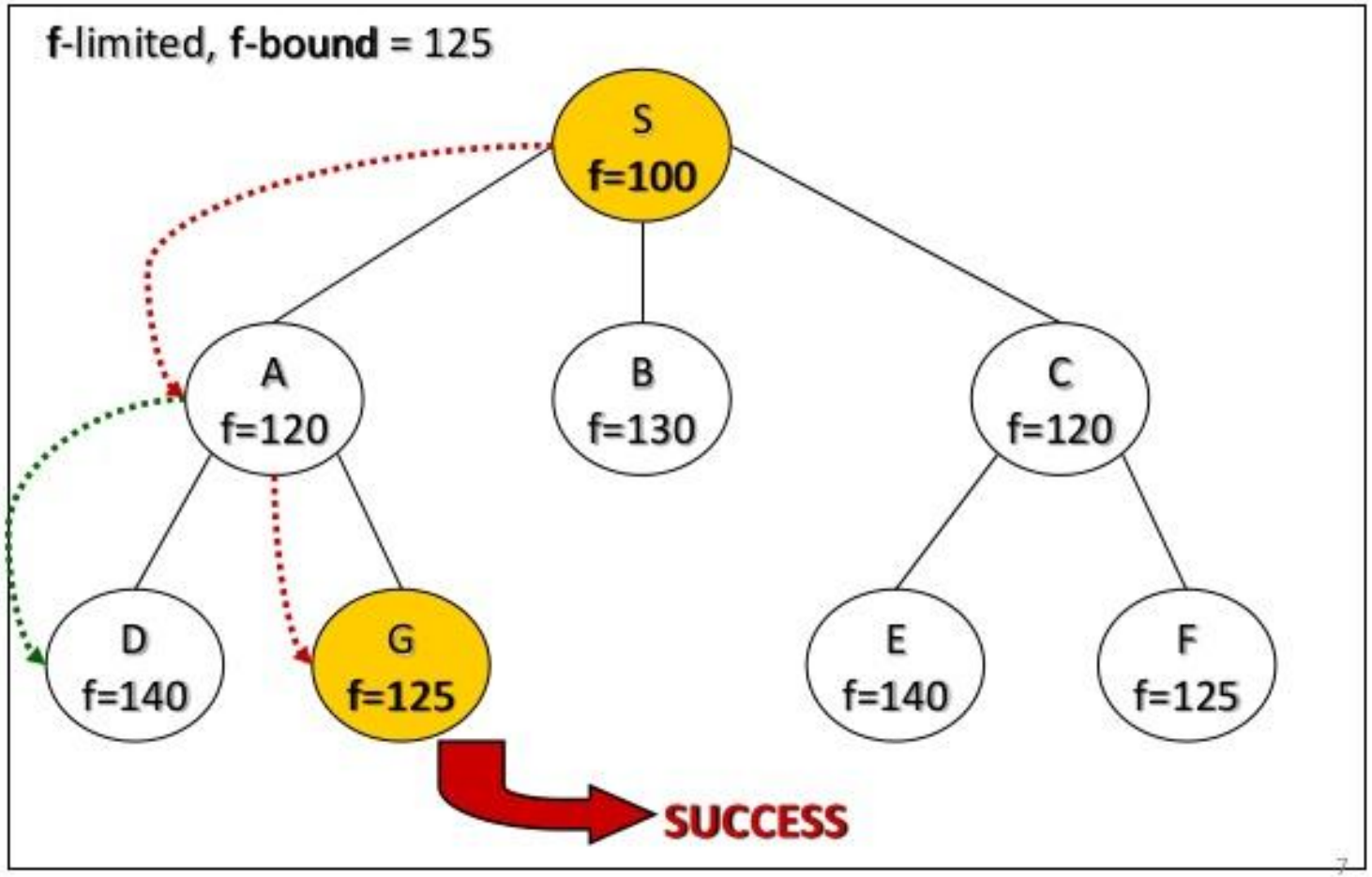
- RBFS is an optimal algorithm if the heuristic function h(n) is admissible.
- Its space complexity is linear in the depth of the deepest optimal solution.
- Its time complexity is rather difficult to characterize: it depends both on the accuracy of the heuristic function and on how often the best path changes as nodes are expanded.
- RBFS suffer from using *too little* memory.
- Because they forget most of what they have done, both algorithms may end up re-expanding the same states many times over.
- It suffer the potentially exponential increase in complexity associated with redundant paths in graphs

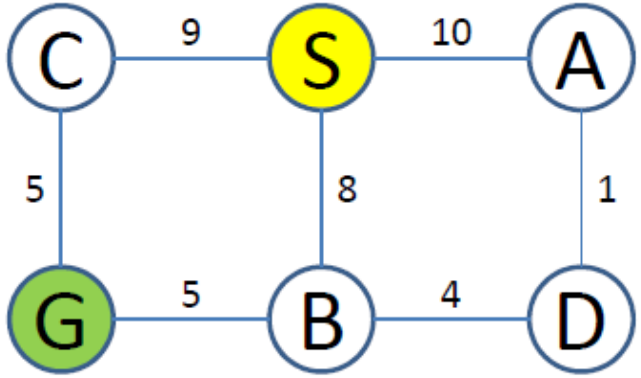
IDA*


Example
f-limited, f-bound = 100
f-new = 120
S
f=100
A
f=120
B
f=130
C
f=120
D
f=140
G
f=125
E
f=140
F
f=125

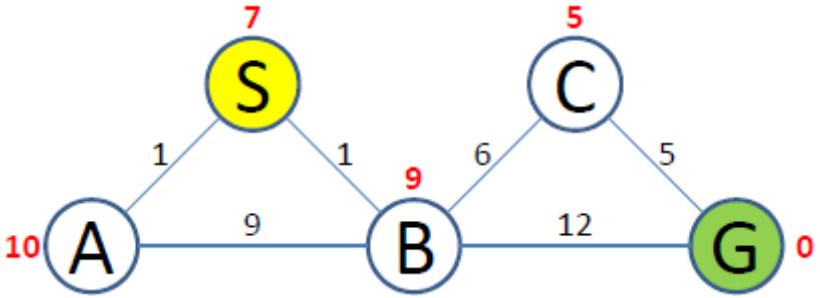
IDA*

Example


f-limited, f-bound = 125
S
f=100
A
f=120
B
f=130
C
f=120
D
f=140
G
f=125
E
f=140
F
f=125
SUCCESS

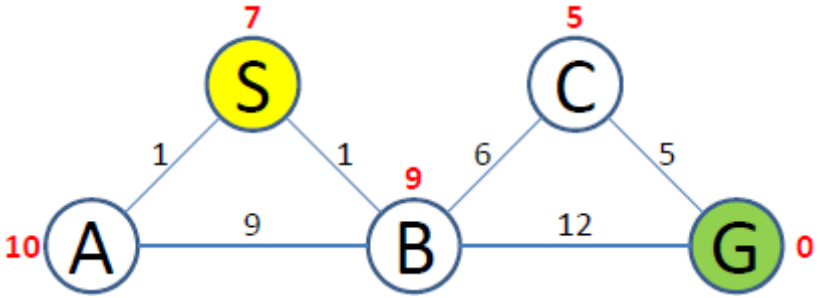


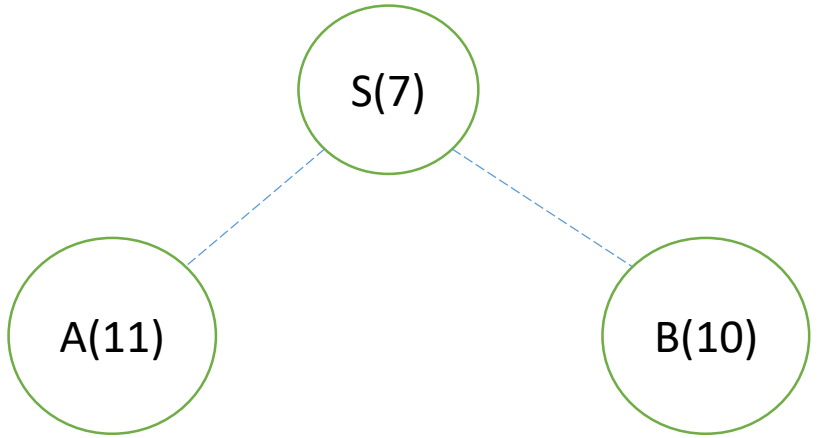
	S	A	B	C	D	G
heuristic	0	0	4	3	0	0



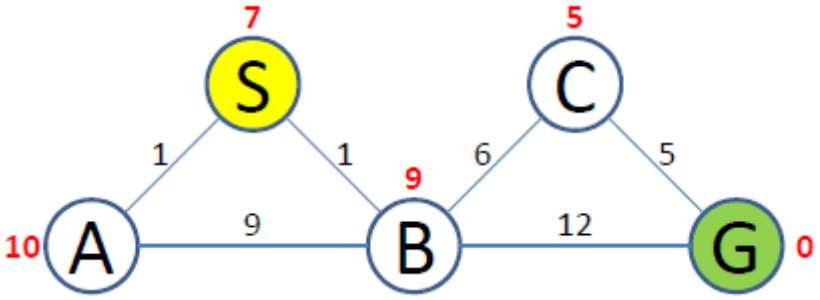
S(7)

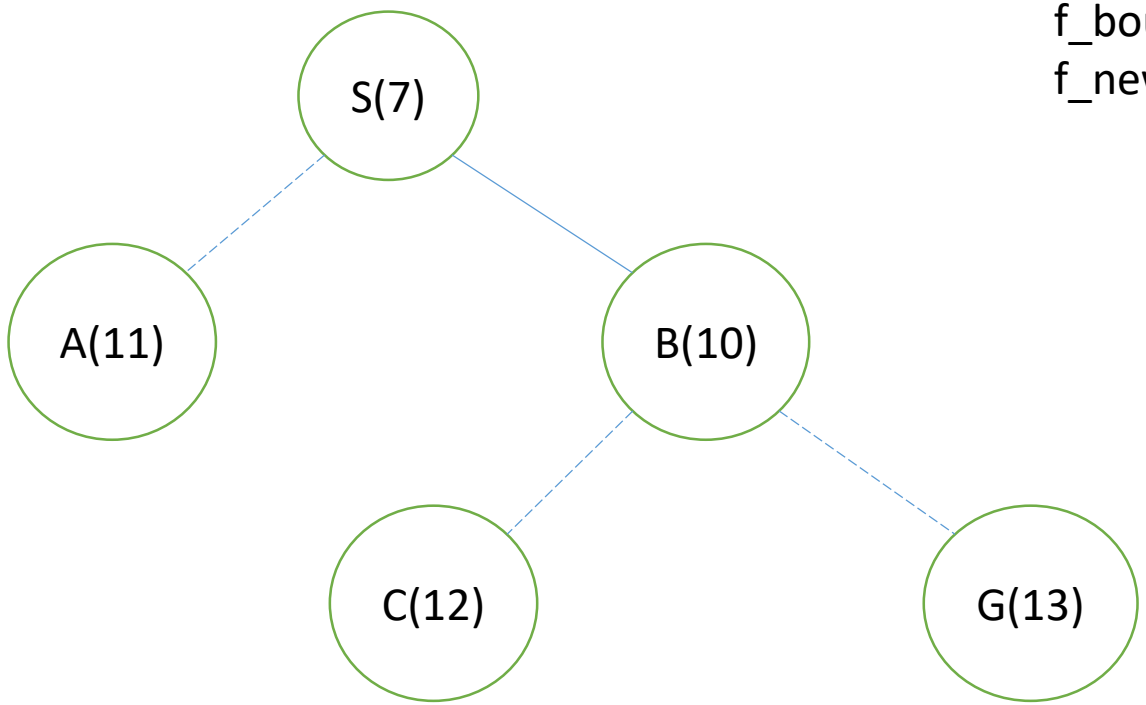
f_bound = 0
f_new = 7



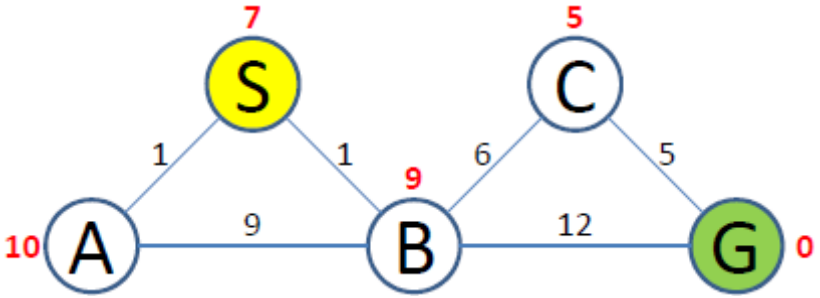


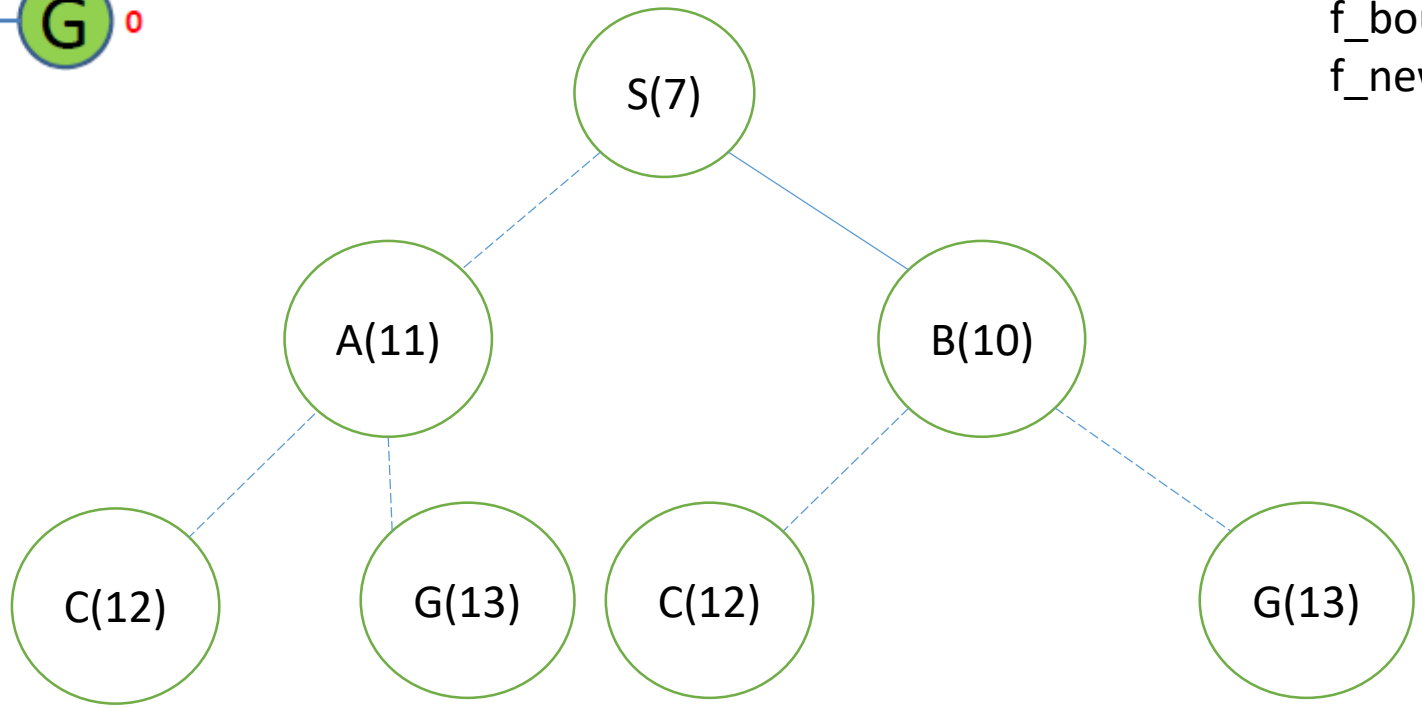
f_bound = 7
f_new = 10





f_bound = 10
f_new = 11





f_bound = 11
f_new =